3_Point_Shooting_Era_Breakdown

September 12, 2025

1 0. *Introduction*

The NBA has been through many different eras, and each came with revolutionary changes to the game. One of the most important eras is today's 3-Point Revolution, which can be traced to the implementation of the radical 3-Point line in 1976. Anyone who has followed the NBA closely over the past decade can see with their eyes the boom of the 3-point shot and how the product on the court is impacted greatly by it when in the decades before that was not the case.

Questions I am trying to solve: 1. When and how did the NBA 3-Points boom arrive withing the past 30 years? 2. What are commanilites in shooting habits that players of todays generation have opposed to their counterparts in the late 1990s and early 2000s?

[3]:
```
# Import packages and models
import os
import kagglehub
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.ticker as mtick
from matplotlib.lines import Line2D
from matplotlib.patches import Circle, Rectangle, Arc
from matplotlib.ticker import PercentFormatter, FuncFormatter
from sklearn.metrics import confusion_matrix, accuracy_score,␣
↪classification_report, ConfusionMatrixDisplay, precision_score,␣
↪recall_score, f1_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score,␣
↪GridSearchCV, RandomizedSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.inspection import permutation_importance
```

2 1. *Data Cleaning and Overview*

In this exploration of 3-Point shooting, I will be implementing different NBA shooting datasets from Kaggle. All datasets used have been cleaned to have no missing/null values.

2.0.1 *Team Stat Dataset:

Majorly compromised of Field Goals made and attempt rates by teams over the course of the seasons from 1999-2023.

[4]:
```
# Importing Team Stats Per Game csv from 'NBA Stats (1947-present)' by Sumitro␣
↪Datta
team_totals = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/nba-dataset/
↪Team Stats Per Game.csv")

# Including seasons from 1999-2023
team_totals = team_totals.loc[(team_totals['season'] < 2025) &␣
↪(team_totals['season'] > 1994)]

# Removing missing values (removes season average)
team_totals.dropna(inplace=True)

# Output summary
print(team_totals.info())
team_totals.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 888 entries, 31 to 947
Data columns (total 28 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   season         888 non-null    int64
 1   lg             888 non-null    object
 2   team           888 non-null    object
 3   abbreviation   888 non-null    object
 4   playoffs       888 non-null    bool
 5   g              888 non-null    float64
 6   mp_per_game    888 non-null    float64
 7   fg_per_game    888 non-null    float64
 8   fga_per_game   888 non-null    float64
 9   fg_percent     888 non-null    float64
 10  x3p_per_game   888 non-null    float64
 11  x3pa_per_game  888 non-null    float64
 12  x3p_percent    888 non-null    float64
 13  x2p_per_game   888 non-null    float64
 14  x2pa_per_game  888 non-null    float64
 15  x2p_percent    888 non-null    float64
 16  ft_per_game    888 non-null    float64
```

```
 17  fta_per_game   888 non-null    float64
 18  ft_percent     888 non-null    float64
 19  orb_per_game   888 non-null    float64
 20  drb_per_game   888 non-null    float64
 21  trb_per_game   888 non-null    float64
 22  ast_per_game   888 non-null    float64
 23  stl_per_game   888 non-null    float64
 24  blk_per_game   888 non-null    float64
 25  tov_per_game   888 non-null    float64
 26  pf_per_game    888 non-null    float64
 27  pts_per_game   888 non-null    float64
dtypes: bool(1), float64(23), int64(1), object(3)
memory usage: 195.1+ KB
None
```

[4]:

```
            season           g  mp_per_game  fg_per_game  fga_per_game  \
count   888.00000  888.000000   888.000000   888.000000    888.000000
mean   2009.64527   79.689189   241.739077    37.745608     82.885698
std       8.61969    6.703262     0.873120     2.626490      4.240888
min    1995.00000   50.000000   240.000000    30.800000     71.200000
25%    2002.00000   82.000000   241.200000    35.900000     79.700000
50%    2010.00000   82.000000   241.800000    37.400000     82.400000
75%    2017.00000   82.000000   242.300000    39.400000     86.000000
max    2024.00000   82.000000   244.900000    47.000000     94.400000

       fg_percent  x3p_per_game  x3pa_per_game  x3p_percent  x2p_per_game  \
count  888.000000    888.000000     888.000000   888.000000    888.000000
mean     0.455257      7.588063      21.236937     0.355268     30.161486
std      0.017396      3.094450       8.333323     0.020537      2.083181
min      0.401000      2.000000       7.400000     0.264000     23.100000
25%      0.443000      5.200000      14.875000     0.343000     28.800000
50%      0.455000      6.700000      18.900000     0.355000     30.250000
75%      0.467000      9.900000      27.200000     0.369000     31.600000
max      0.512000     16.700000      45.400000     0.428000     36.300000

       …  ft_percent  orb_per_game  drb_per_game  trb_per_game  \
count  …  888.000000    888.000000    888.000000    888.000000
mean   …    0.757423     11.295495     31.265428     42.558221
std    …    0.031017      1.543449      2.407787      2.138976
min    …    0.660000      7.600000     24.900000     35.600000
25%    …    0.739000     10.275000     29.500000     41.100000
50%    …    0.759000     11.200000     31.100000     42.500000
75%    …    0.779000     12.300000     33.000000     44.000000
max    …    0.839000     18.500000     42.200000     51.700000

       ast_per_game  stl_per_game  blk_per_game  tov_per_game  pf_per_game  \
count    888.000000    888.000000    888.000000    888.000000   888.000000
```

```
mean      22.472410      7.695495      4.925225     14.604617    21.123536
std        2.352488      0.897172      0.829702      1.254184     1.883219
min       15.600000      5.500000      2.400000     11.100000    15.600000
25%       20.800000      7.100000      4.300000     13.800000    19.800000
50%       22.200000      7.600000      4.900000     14.600000    21.000000
75%       23.900000      8.200000      5.400000     15.300000    22.400000
max       30.800000     12.000000      8.200000     19.000000    27.100000

       pts_per_game
count    888.000000
mean     101.342117
std        7.573505
min       81.900000
25%       95.775000
50%       99.950000
75%      106.225000
max      123.300000

[8 rows x 24 columns]
```

[5]:
```python
# Displays the data and timeline
team_totals
```

[5]:
```
     season   lg                 team abbreviation  playoffs     g  \
31     2024  NBA        Atlanta Hawks          ATL     False  82.0
32     2024  NBA       Boston Celtics          BOS      True  82.0
33     2024  NBA        Brooklyn Nets          BRK     False  82.0
34     2024  NBA        Chicago Bulls          CHI     False  82.0
35     2024  NBA    Charlotte Hornets          CHO     False  82.0
..      …  …                  …          …       …   …
943    1995  NBA     Sacramento Kings          SAC     False  82.0
944    1995  NBA    San Antonio Spurs          SAS      True  82.0
945    1995  NBA  Seattle SuperSonics          SEA      True  82.0
946    1995  NBA            Utah Jazz          UTA      True  82.0
947    1995  NBA   Washington Bullets          WSB     False  82.0

     mp_per_game  fg_per_game  fga_per_game  fg_percent  …  ft_percent  \
31         242.1         43.0          92.5       0.465  …       0.797
32         241.8         43.9          90.2       0.487  …       0.807
33         241.5         40.7          89.1       0.456  …       0.756
34         243.7         42.0          89.5       0.470  …       0.791
35         240.6         40.0          87.0       0.460  …       0.786
..           …          …           …          …  …         …
943        242.1         36.9          78.8       0.468  …       0.711
944        242.1         39.5          81.5       0.484  …       0.738
945        241.8         40.4          82.2       0.491  …       0.758
946        241.2         39.5          77.3       0.512  …       0.781
```

```
947        242.1         38.7          84.1       0.460  …       0.724

     orb_per_game  drb_per_game  trb_per_game  ast_per_game  stl_per_game  \
31           12.5          32.2          44.7          26.6           7.5
32           10.7          35.6          46.3          26.9           6.8
33           11.4          32.6          44.1          25.6           6.8
34           11.2          32.6          43.8          25.0           7.8
35            9.3          31.0          40.3          24.8           6.9
..            …            …            …            …            …
943          13.1          28.4          41.4          22.2           7.9
944          12.5          32.5          45.0          23.4           8.0
945          13.0          28.5          41.5          25.8          11.2
946          10.7          29.4          40.1          27.5           9.2
947          12.7          27.1          39.8          21.3           7.9

     blk_per_game  tov_per_game  pf_per_game  pts_per_game
31            4.5          13.5         18.6         118.3
32            6.6          11.9         16.2         120.6
33            5.2          13.1         18.5         110.4
34            4.8          12.2         18.8         112.3
35            4.5          13.8         18.0         106.6
..            …            …            …            …
943           5.6          17.7         24.9          98.2
944           5.6          15.2         22.8         106.6
945           4.8          15.8         25.2         110.4
946           4.8          15.7         24.9         106.4
947           4.9          15.9         23.8         100.5

[888 rows x 28 columns]
```

2.0.2 Individual Player Shooting Dataset:

Focuses on providing player shooting rates from different distances on the court.

[6]:
```
# Importing Player Shooting csv from 'NBA Stats (1947-present)'

player_shooting = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/
↪nba-dataset/Player Shooting.csv")

# Drop cols that do not have any application for my analysis and cols with␣
↪majority NA values
player_shooting.drop(['num_heaves_made', 'num_heaves_attempted', 'num_of_dunks',
                      'percent_dunks_of_fga', 'percent_assisted_x2p_fg',␣
↪'percent_assisted_x3p_fg',
                      'percent_corner_3s_of_3pa', 'corner_3_point_percent'],␣
↪axis=1, inplace=True)
```

```python
# Clean the position column for players marked as double position labels (i.e␣
↪"SG-PG")
player_shooting['pos'] = player_shooting['pos'].str.replace('-SG', '')
player_shooting['pos'] = player_shooting['pos'].str.replace('-SF', '')
player_shooting['pos'] = player_shooting['pos'].str.replace('-C', '')
player_shooting['pos'] = player_shooting['pos'].str.replace('-PF', '')
player_shooting['pos'] = player_shooting['pos'].str.replace('-PG', '')
player_shooting['pos'] = player_shooting['pos'].str.replace('-PG-SF', '')

# Clarify the shorthanded column names
player_shooting.rename(columns = {'pos': 'position', 'lg':'league', 'tm':'team',
                                  'g':'game in a season', 'mp':'minutes␣
↪played'},
                       inplace=True)

# Have to match our previous data to only include 1995-2025
player_shooting = player_shooting.loc[(player_shooting['season'] < 2025) &␣
↪(player_shooting['season'] > 1994)]

# Drop the NaN values/missing values
player_shooting.dropna(inplace=True)

# Output summary
print(player_shooting.info())
player_shooting.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 12968 entries, 735 to 17519
Data columns (total 24 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   season                        12968 non-null  int64
 1   league                        12968 non-null  object
 2   player                        12968 non-null  object
 3   player_id                     12968 non-null  object
 4   age                           12968 non-null  int64
 5   team                          12968 non-null  object
 6   position                      12968 non-null  object
 7   game in a season              12968 non-null  int64
 8   gs                            12968 non-null  int64
 9   minutes played                12968 non-null  int64
 10  fg_percent                    12968 non-null  float64
 11  avg_dist_fga                  12968 non-null  float64
 12  percent_fga_from_x2p_range    12968 non-null  float64
 13  percent_fga_from_x0_3_range   12968 non-null  float64
 14  percent_fga_from_x3_10_range  12968 non-null  float64
 15  percent_fga_from_x10_16_range 12968 non-null  float64
```

```
 16  percent_fga_from_x16_3p_range  12968 non-null  float64
 17  percent_fga_from_x3p_range     12968 non-null  float64
 18  fg_percent_from_x2p_range      12968 non-null  float64
 19  fg_percent_from_x0_3_range     12968 non-null  float64
 20  fg_percent_from_x3_10_range    12968 non-null  float64
 21  fg_percent_from_x10_16_range   12968 non-null  float64
 22  fg_percent_from_x16_3p_range   12968 non-null  float64
 23  fg_percent_from_x3p_range      12968 non-null  float64
dtypes: float64(14), int64(5), object(5)
memory usage: 2.5+ MB
None
```

[6]:

```
             season           age  game in a season            gs  \
count  12968.000000  12968.000000      12968.000000  12968.000000
mean    2011.463757     26.744294         51.957202     25.533236
std        8.118440      4.263150         22.970656     27.942324
min     1997.000000     18.000000          1.000000      0.000000
25%     2005.000000     23.000000         32.000000      1.000000
50%     2012.000000     26.000000         56.000000     13.000000
75%     2019.000000     30.000000         73.000000     48.000000
max     2024.000000     43.000000         85.000000     83.000000

       minutes played    fg_percent  avg_dist_fga  percent_fga_from_x2p_range  \
count    12968.000000  12968.000000  12968.000000                12968.000000
mean      1249.997918      0.436192     13.348897                    0.711078
std        842.647006      0.065943      4.197982                    0.202982
min         12.000000      0.000000      1.000000                    0.063000
25%        509.000000      0.400000     10.600000                    0.563000
50%       1145.000000      0.435000     13.800000                    0.707000
75%       1904.000000      0.472000     16.400000                    0.899000
max       3485.000000      0.765000     27.500000                    0.999000

       percent_fga_from_x0_3_range  percent_fga_from_x3_10_range  \
count                 12968.000000                  12968.000000
mean                      0.277575                      0.150624
std                       0.143254                      0.086977
min                       0.003000                      0.004000
25%                       0.174000                      0.085000
50%                       0.254000                      0.134000
75%                       0.353000                      0.200000
max                       0.895000                      0.618000

       percent_fga_from_x10_16_range  percent_fga_from_x16_3p_range  \
count                   12968.000000                   12968.000000
mean                       0.107889                       0.174848
std                        0.065361                       0.122288
min                        0.002000                       0.001000
```

```
25%                         0.061000                      0.076000
50%                         0.095000                      0.156000
75%                         0.143000                      0.252000
max                         0.532000                      0.783000

       percent_fga_from_x3p_range  fg_percent_from_x2p_range  \
count                12968.000000               12968.000000
mean                     0.288926                   0.473027
std                      0.202981                   0.074655
min                      0.001000                   0.000000
25%                      0.101000                   0.432000
50%                      0.293000                   0.473000
75%                      0.437000                   0.515000
max                      0.937000                   0.857000

       fg_percent_from_x0_3_range  fg_percent_from_x3_10_range  \
count                12968.000000                 12968.000000
mean                     0.608928                    0.367447
std                      0.118745                    0.142960
min                      0.000000                    0.000000
25%                      0.549000                    0.300000
50%                      0.615000                    0.378000
75%                      0.676000                    0.444000
max                      1.000000                    1.000000

       fg_percent_from_x10_16_range  fg_percent_from_x16_3p_range  \
count                  12968.000000                  12968.000000
mean                       0.363713                      0.373387
std                        0.157913                      0.149239
min                        0.000000                      0.000000
25%                        0.292000                      0.318750
50%                        0.375000                      0.385000
75%                        0.444000                      0.444000
max                        1.000000                      1.000000

       fg_percent_from_x3p_range
count               12968.000000
mean                    0.293024
std                     0.147914
min                     0.000000
25%                     0.250000
50%                     0.333000
75%                     0.376000
max                     1.000000
```

[7]:
```
# Display data
player_shooting
```

[7]:

```
       season league            player  player_id  age team position  \
735      2024    NBA  Precious Achiuwa  achiupr01   24  2TM       PF
736      2024    NBA  Precious Achiuwa  achiupr01   24  TOR        C
737      2024    NBA  Precious Achiuwa  achiupr01   24  NYK       PF
738      2024    NBA       Bam Adebayo  adebaba01   26  MIA        C
739      2024    NBA      Ochai Agbaji  agbajoc01   23  2TM       SG
…         …     …                 …         …   …   …       …
17514    1997    NBA        David Wood   woodda01   32  MIL       PF
17516    1997    NBA   Lorenzen Wright  wrighlo02   21  LAC        C
17517    1997    NBA    Sharone Wright  wrighsh01   24  TOR        C
17518    1997    NBA      George Zídek  zidekge01   23  2TM        C
17519    1997    NBA      George Zídek  zidekge01   23  CHH        C

       game in a season  gs  minutes played  …  \
735                  74  18            1624  …
736                  25   0             437  …
737                  49  18            1187  …
738                  71  71            2416  …
739                  78  28            1641  …
…                 …  ..              …  …
17514                46   0             240  …
17516                77  51            1936  …
17517                60  28            1009  …
17518                52   2             376  …
17519                36   2             288  …

       percent_fga_from_x3_10_range  percent_fga_from_x10_16_range  \
735                           0.215                          0.047
736                           0.247                          0.047
737                           0.197                          0.047
738                           0.367                          0.278
739                           0.176                          0.037
…                             …                            …
17514                         0.158                          0.105
17516                         0.299                          0.185
17517                         0.372                          0.171
17518                         0.297                          0.127
17519                         0.188                          0.141

       percent_fga_from_x16_3p_range  percent_fga_from_x3p_range  \
735                            0.011                       0.207
736                            0.012                       0.276
737                            0.010                       0.167
738                            0.069                       0.041
739                            0.012                       0.487
…                              …                         …
17514                          0.053                       0.395
```

```
17516                          0.049                       0.008
17517                          0.027                       0.002
17518                          0.212                       0.017
17519                          0.200                       0.024

       fg_percent_from_x2p_range  fg_percent_from_x0_3_range  \
735                        0.562                       0.697
736                        0.528                       0.662
737                        0.578                       0.711
738                        0.528                       0.727
739                        0.523                       0.664
…                          …                         …
17514                      0.652                       0.909
17516                      0.483                       0.533
17517                      0.400                       0.512
17518                      0.422                       0.220
17519                      0.398                       0.211

       fg_percent_from_x3_10_range  fg_percent_from_x10_16_range  \
735                          0.317                         0.273
736                          0.357                         0.250
737                          0.288                         0.286
738                          0.515                         0.406
739                          0.329                         0.438
…                            …                           …
17514                        0.333                         0.250
17516                        0.483                         0.385
17517                        0.300                         0.333
17518                        0.457                         0.600
17519                        0.500                         0.583

       fg_percent_from_x16_3p_range  fg_percent_from_x3p_range
735                           0.200                      0.268
736                           0.500                      0.277
737                           0.000                      0.260
738                           0.386                      0.357
739                           0.200                      0.294
…                             …                        …
17514                         1.000                      0.333
17516                         0.375                      0.250
17517                         0.455                      0.000
17518                         0.600                      0.000
17519                         0.588                      0.000

[12968 rows x 24 columns]
```

2.0.3 Positional Shooting Datasets:

These collection of datasets provide positional data of everyshot attempted by every player in the NBA within the specified season.

[8]:
```
from re import S
# Import a few shot location datasets for the years we are observing
shots_04 = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/nba-shots-data/
 ↪NBA_2004_Shots.csv')
shots_07 = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/nba-shots-data/
 ↪NBA_2007_Shots.csv')
shots_11 = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/nba-shots-data/
 ↪NBA_2011_Shots.csv')
shots_15 = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/nba-shots-data/
 ↪NBA_2015_Shots.csv')
shots_19 = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/nba-shots-data/
 ↪NBA_2019_Shots.csv')
shots_23 = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/nba-shots-data/
 ↪NBA_2023_Shots.csv')

# list of all our df to make it easier to clean and manipulate
all_years = [shots_04, shots_07, shots_11, shots_15, shots_19, shots_23]

# Align decimal place in x and y coords to fit our shot data
for df in all_years:
  df[['LOC_X', 'LOC_Y']] = (df[['LOC_X', 'LOC_Y']]*10)

# Use Feature Encoding on the SHOT_MADE column for our ML model later
for df in all_years:
  le = LabelEncoder()
  label = le.fit_transform(df['SHOT_MADE'])
  label
  df.loc[:,'SHOT_MADE'] = label

# Move the 'SHOT_MADE' column to the end of each df to use in ML models later
for df in all_years:
  df.insert(len(df.columns)-1, 'SHOT_MADE', df.pop('SHOT_MADE'))

# Drop irrelevant cols for our analysis
for df in all_years:
  df.drop(['SEASON_1', 'TEAM_ID', 'POSITION_GROUP', 'POSITION',
       'GAME_DATE', 'GAME_ID','HOME_TEAM', 'AWAY_TEAM'], axis=1, inplace=True)

# Simplifing Action_Type value names
jumper = ['Fadeaway Jump Shot', 'Turnaround Jump Shot', 'Running Jump Shot',␣
 ↪'Jump Bank Shot', 'Hook Bank Shot',
```

```
          'Jump Hook Shot', 'Running Hook Shot', 'Hook Shot', 'Turnaround Hook␣
 ↪Shot', 'Driving Hook Shot',
          'Driving Jump Shot']

layup = ['Driving Layup Shot', 'Driving Finger Roll Shot',  'Running Layup␣
 ↪Shot',
         'Reverse Layup Shot', 'Running Finger Roll Shot', 'Alley Oop Layup␣
 ↪shot', 'Finger Roll Shot',
       'Turnaround Finger Roll Shot', 'Running Tip Shot', 'Tip Shot',␣
 ↪'Turnaround Layup Shot']

dunk = ['Alley Oop Dunk Shot', 'Slam Dunk Shot', 'Driving Dunk Shot', 'Reverse␣
 ↪Dunk Shot',
        'Follow Up Dunk Shot', 'Running Dunk Shot']

for a in jumper:
    shots_04['ACTION_TYPE'] = shots_04['ACTION_TYPE'].str.replace(str(a), 'Jump␣
 ↪Shot')

for b in layup:
    shots_04['ACTION_TYPE'] = shots_04['ACTION_TYPE'].str.replace(str(b),␣
 ↪'Layup Shot')

for c in dunk:
    shots_04['ACTION_TYPE'] = shots_04['ACTION_TYPE'].str.replace(str(c), 'Dunk␣
 ↪Shot')

for a in jumper:
    shots_07['ACTION_TYPE'] = shots_07['ACTION_TYPE'].str.replace(str(a), 'Jump␣
 ↪Shot')

for b in layup:
    shots_07['ACTION_TYPE'] = shots_07['ACTION_TYPE'].str.replace(str(b),␣
 ↪'Layup Shot')

for c in dunk:
    shots_07['ACTION_TYPE'] = shots_07['ACTION_TYPE'].str.replace(str(c), 'Dunk␣
 ↪Shot')

for a in jumper:
    shots_11['ACTION_TYPE'] = shots_11['ACTION_TYPE'].str.replace(str(a), 'Jump␣
 ↪Shot')

for b in layup:
    shots_11['ACTION_TYPE'] = shots_11['ACTION_TYPE'].str.replace(str(b),␣
 ↪'Layup Shot')
```

```
for c in dunk:
    shots_11['ACTION_TYPE'] = shots_11['ACTION_TYPE'].str.replace(str(c), 'Dunk␣
 ↪Shot')

for a in jumper:
    shots_15['ACTION_TYPE'] = shots_15['ACTION_TYPE'].str.replace(str(a), 'Jump␣
 ↪Shot')

for b in layup:
    shots_15['ACTION_TYPE'] = shots_15['ACTION_TYPE'].str.replace(str(b),␣
 ↪'Layup Shot')

for c in dunk:
    shots_15['ACTION_TYPE'] = shots_15['ACTION_TYPE'].str.replace(str(c), 'Dunk␣
 ↪Shot')

for a in jumper:
    shots_19['ACTION_TYPE'] = shots_19['ACTION_TYPE'].str.replace(str(a), 'Jump␣
 ↪Shot')

for b in layup:
    shots_19['ACTION_TYPE'] = shots_19['ACTION_TYPE'].str.replace(str(b),␣
 ↪'Layup Shot')

for c in dunk:
    shots_19['ACTION_TYPE'] = shots_19['ACTION_TYPE'].str.replace(str(c), 'Dunk␣
 ↪Shot')

for a in jumper:
    shots_23['ACTION_TYPE'] = shots_23['ACTION_TYPE'].str.replace(str(a), 'Jump␣
 ↪Shot')

for b in layup:
    shots_23['ACTION_TYPE'] = shots_23['ACTION_TYPE'].str.replace(str(b),␣
 ↪'Layup Shot')

for c in dunk:
    shots_23['ACTION_TYPE'] = shots_23['ACTION_TYPE'].str.replace(str(c), 'Dunk␣
 ↪Shot')


# All df have the same form, displaying just one
print(shots_04.info())
shots_04.describe()
```

```
/tmp/ipython-input-2482113987.py:22: FutureWarning: Setting an item of
```

```
incompatible dtype is deprecated and will raise in a future error of pandas.
Value '[1 1 0 … 0 0 0]' has dtype incompatible with bool, please explicitly
cast to a compatible dtype first.
  df.loc[:,'SHOT_MADE'] = label
/tmp/ipython-input-2482113987.py:22: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise in a future error of pandas.
Value '[1 0 0 … 0 1 0]' has dtype incompatible with bool, please explicitly
cast to a compatible dtype first.
  df.loc[:,'SHOT_MADE'] = label
/tmp/ipython-input-2482113987.py:22: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise in a future error of pandas.
Value '[0 0 1 … 1 0 0]' has dtype incompatible with bool, please explicitly
cast to a compatible dtype first.
  df.loc[:,'SHOT_MADE'] = label
/tmp/ipython-input-2482113987.py:22: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise in a future error of pandas.
Value '[1 1 1 … 0 0 0]' has dtype incompatible with bool, please explicitly
cast to a compatible dtype first.
  df.loc[:,'SHOT_MADE'] = label
/tmp/ipython-input-2482113987.py:22: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise in a future error of pandas.
Value '[1 1 1 … 1 0 0]' has dtype incompatible with bool, please explicitly
cast to a compatible dtype first.
  df.loc[:,'SHOT_MADE'] = label
/tmp/ipython-input-2482113987.py:22: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise in a future error of pandas.
Value '[0 1 1 … 1 1 0]' has dtype incompatible with bool, please explicitly
cast to a compatible dtype first.
  df.loc[:,'SHOT_MADE'] = label
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 189803 entries, 0 to 189802
Data columns (total 18 columns):
 #   Column         Non-Null Count   Dtype
---  ------         --------------   -----
 0   SEASON_2       189803 non-null  object
 1   TEAM_NAME      189803 non-null  object
 2   PLAYER_ID      189803 non-null  int64
 3   PLAYER_NAME    189803 non-null  object
 4   EVENT_TYPE     189803 non-null  object
 5   ACTION_TYPE    189803 non-null  object
 6   SHOT_TYPE      189803 non-null  object
 7   BASIC_ZONE     189803 non-null  object
 8   ZONE_NAME      189803 non-null  object
 9   ZONE_ABB       189803 non-null  object
 10  ZONE_RANGE     189803 non-null  object
 11  LOC_X          189803 non-null  float64
 12  LOC_Y          189803 non-null  float64
```

```
 13  SHOT_DISTANCE  189803 non-null  int64
 14  QUARTER        189803 non-null  int64
 15  MINS_LEFT      189803 non-null  int64
 16  SECS_LEFT      189803 non-null  int64
 17  SHOT_MADE      189803 non-null  int64
dtypes: float64(2), int64(6), object(10)
memory usage: 26.1+ MB
None
```

[8]:

```
           PLAYER_ID          LOC_X          LOC_Y  SHOT_DISTANCE  \
count  189803.000000  189803.000000  189803.000000  189803.000000
mean     1530.123844       2.739999     119.843994      11.595955
std       757.888105     111.105957      82.611039       9.546158
min        15.000000    -250.000000       0.500000       0.000000
25%       947.000000     -61.000000      52.500000       0.000000
50%      1730.000000       0.000000      83.500000      13.000000
75%      2200.000000      71.000000     182.500000      19.000000
max      2724.000000     248.000000     908.500000      87.000000

             QUARTER      MINS_LEFT      SECS_LEFT      SHOT_MADE
count  189803.000000  189803.000000  189803.000000  189803.000000
mean        2.475388       5.333588      28.669547       0.438639
std         1.138648       3.472361      17.466134       0.496222
min         1.000000       0.000000       0.000000       0.000000
25%         1.000000       2.000000      14.000000       0.000000
50%         2.000000       5.000000      29.000000       0.000000
75%         3.000000       8.000000      44.000000       1.000000
max         7.000000      12.000000      59.000000       1.000000
```

[9]:

```
shots_04
```

[9]:

```
        SEASON_2               TEAM_NAME  PLAYER_ID       PLAYER_NAME  \
0       2003-04      Los Angeles Lakers        977       Kobe Bryant
1       2003-04  Portland Trail Blazers        757  Damon Stoudamire
2       2003-04      Los Angeles Lakers        977       Kobe Bryant
3       2003-04  Portland Trail Blazers        757  Damon Stoudamire
4       2003-04  Portland Trail Blazers        757  Damon Stoudamire
…         …                      …        …                 …
189798  2003-04      Philadelphia 76ers       2422      John Salmons
189799  2003-04       San Antonio Spurs       1938     Manu Ginobili
189800  2003-04      Los Angeles Lakers        406  Shaquille O'Neal
189801  2003-04            Phoenix Suns       2063      Jake Voskuhl
189802  2003-04              Miami Heat       2548       Dwyane Wade

         EVENT_TYPE ACTION_TYPE       SHOT_TYPE             BASIC_ZONE  \
0         Made Shot   Jump Shot  3PT Field Goal      Above the Break 3
1         Made Shot  Layup Shot  2PT Field Goal        Restricted Area
```

```
2       Missed Shot   Jump Shot  2PT Field Goal              Mid-Range
3         Made Shot   Jump Shot  2PT Field Goal              Mid-Range
4       Missed Shot   Jump Shot  2PT Field Goal              Mid-Range
…             …         …             …                      …
189798    Made Shot   Jump Shot  3PT Field Goal          Left Corner 3
189799  Missed Shot   Jump Shot  3PT Field Goal      Above the Break 3
189800  Missed Shot   Jump Shot  2PT Field Goal  In The Paint (Non-RA)
189801  Missed Shot  Layup Shot  2PT Field Goal        Restricted Area
189802  Missed Shot  Layup Shot  2PT Field Goal        Restricted Area

               ZONE_NAME ZONE_ABB       ZONE_RANGE  LOC_X  LOC_Y  \
0       Left Side Center       LC          24+ ft.  200.0  213.5
1                Center        C  Less Than 8 ft.   -0.0   52.5
2       Left Side Center       LC        16-24 ft.  133.0  244.5
3              Left Side        L        16-24 ft.  164.0  139.5
4             Right Side        R        16-24 ft. -158.0   78.5
…                  …      …               …    …    …
189798         Left Side        L          24+ ft.  233.0   95.5
189799  Left Side Center       LC          24+ ft.  151.0  258.5
189800           Center        C  Less Than 8 ft.  -47.0   98.5
189801           Center        C  Less Than 8 ft.  -23.0   60.5
189802           Center        C  Less Than 8 ft.   -0.0   52.5

        SHOT_DISTANCE  QUARTER  MINS_LEFT  SECS_LEFT  SHOT_MADE
0                  25        6          0          0          1
1                   0        6          0          2          1
2                  23        6          0          9          0
3                  18        6          0         31          1
4                  16        6          0         55          0
…                 …      …         …         …         …
189798             23        1         11         21          1
189799             25        1         11         24          0
189800              6        1         11         39          0
189801              2        1         11         42          0
189802              0        1         11         48          0

[189803 rows x 18 columns]
```

3 2. *Shooting Breakdown*

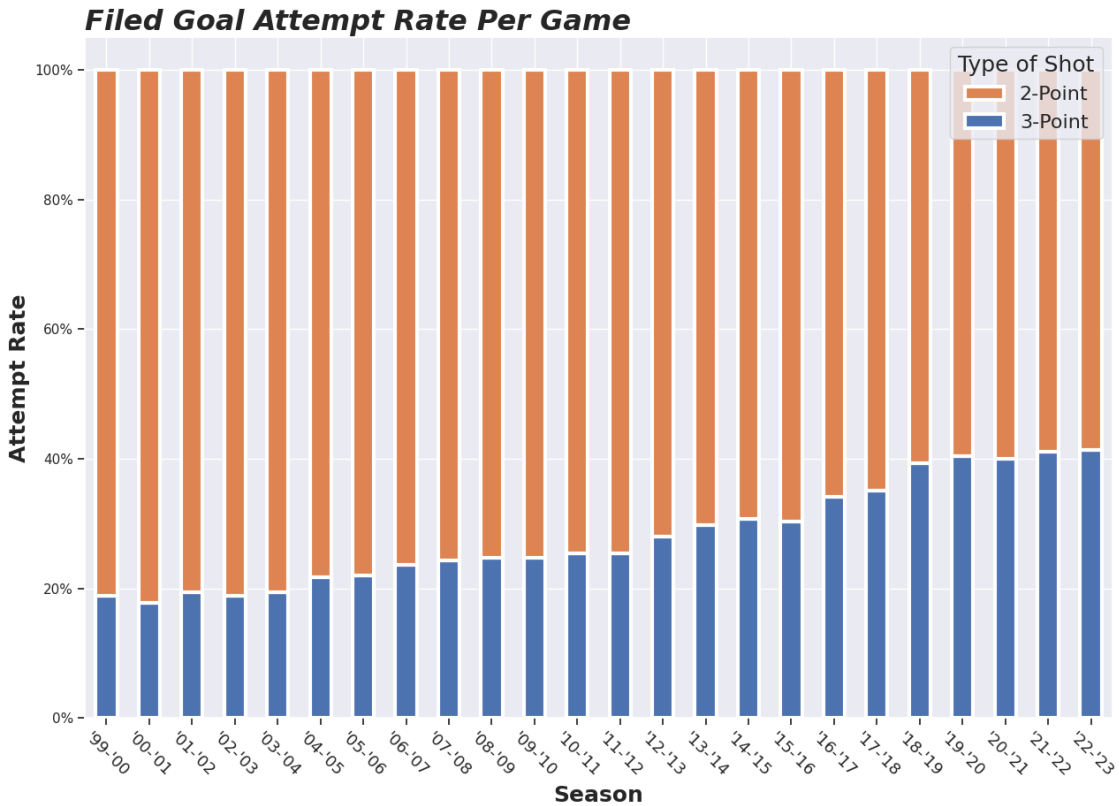
3.1 2.1 3-Pointer vs 2-Pointer

[10]:
```
# Breaking down the shooting between 3PT and 2PT
# First set up the plot
sns.set(rc={'figure.figsize':(15,10)})
sns.color_palette("tab10")
```

```
attempted = player_shooting.groupby('season')[['percent_fga_from_x3p_range',␣
↪'percent_fga_from_x2p_range']].mean().reset_index()

# Filter attempted DataFrame to match the number of labels
attempted = attempted.loc[(attempted['season'] > 1999) & (attempted['season'] <␣
↪2024)]


# Plotting the values
ax = attempted.set_index('season').plot(kind='bar',
                                        stacked=True,
                                        edgecolor='white',
                                        linewidth=3)
# Creating appropriate Titles
ax.axes.set_title('Filed Goal Attempt Rate Per Game',
                  fontsize=23,
                  loc='left',
                  fontweight='bold',
                  fontstyle='oblique')
ax.set_xlabel("Season",
              fontsize=18,
              fontweight='bold')
ax.set_ylabel("Attempt Rate",
              fontsize=18,
              fontweight='bold')

# Format the ytick labels to be a percentage opposed to decimals
ax.yaxis.set_major_formatter(FuncFormatter(lambda y,_:'{:.0%}'.format(y)))

# Format the xtick labels
ax.set_xticks(range(len(attempted.set_index('season'))))
ax.set_xticklabels(["'99-'00", "'00-'01", "'01-'02", "'02-'03",
                    "'03-'04", "'04-'05", "'05-'06", "'06-'07",
                    "'07-'08", "'08-'09", "'09-'10", "'10-'11",
                    "'11-'12", "'12-'13", "'13-'14", "'14-'15",
                    "'15-'16", "'16-'17", "'17-'18", "'18-'19",
                    "'19-'20", "'20-'21", "'21-'22", "'22-'23"],
                   rotation=-45,
                   size=13)
ax.tick_params(bottom=True,
               left=True)
# Configuring the legend
handles, labels = ax.get_legend_handles_labels()
ax.legend(list(reversed(handles)),
          list(reversed(['3-Point', '2-Point'])),
          title='Type of Shot',
          loc='upper right',
```

```
            title_fontsize=18,
            fontsize = 16)
```

[10]: <matplotlib.legend.Legend at 0x7fb547071d00>



As shown by the plot, withing the last 30 years, we see a consistent growth of 3-Pointers being attempted per game by players. From 1 out of every 5 shots in the present-day NBA being a 3.

This might not be a revolutionary jump in shooting rates but it is a staggering increase, especially for on-lookers of the sport who felt the wave of changer in player output with such things as ppg (points per game) or FGP (field goal percentages)

4 2.2 *Are Players Making 3-Pointers?*

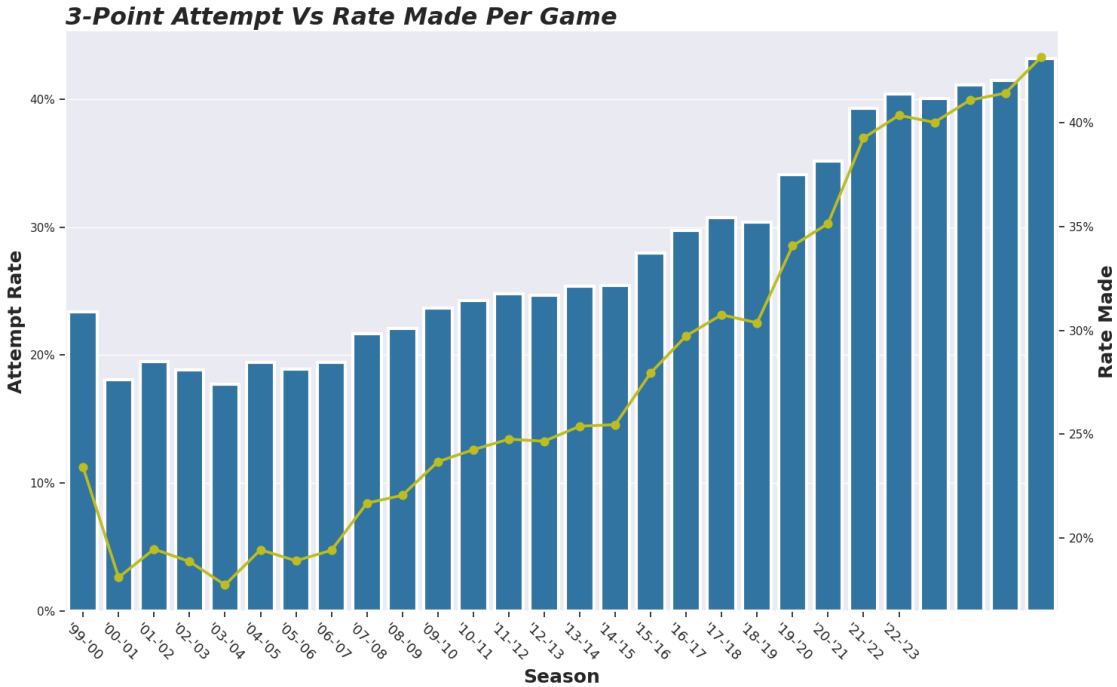
Focusing on Field Goals (FG) percentage, did this 3-Point boom have a positive effect on how well players were shooting the 3 ball?

If so, how much of an an impact did it have on the total score, individual player Points per game, and overall success to a teams win percentage?

[11]:
```
# Setting up the plot
fig, ax = plt.subplots(figsize=(17,10))
```

```
sns.set_theme(style='dark',palette='binary')

# Plotting
g = sns.barplot(data=player_shooting,
                x='season',
                y='percent_fga_from_x3p_range',
                linewidth=3,
                errorbar=None,
                ax=ax,
                color='tab:blue',
                edgecolor='white')
ax2 = ax.twinx()
g = sns.pointplot(data=player_shooting,
                  x='season',
                  y='percent_fga_from_x3p_range',
                  errorbar=None,
                  ax=ax2,
                  color='tab:olive')
# Titles
ax.set_title('3-Point Attempt Vs Rate Made Per Game',
             fontsize=23,
             loc='left',
             fontweight='bold',
             fontstyle='oblique')
ax.set_xlabel("Season",
              fontsize=18,
              fontweight='bold')
ax.set_ylabel("Attempt Rate",
              fontsize=18,
              fontweight='bold')
ax2.set_ylabel("Rate Made",
              fontsize=18,
              fontweight='bold')

# Formatting the y labels to be percentages opposed to decimals
ax.yaxis.set_major_formatter(FuncFormatter(lambda y,_:'{:.0%}'.format(y)))
ax2.yaxis.set_major_formatter(FuncFormatter(lambda y,_:'{:.0%}'.format(y)))

# Formatting x label
ax.set_xticks(range(24))
ax.set_xticklabels(["'99-'00", "'00-'01", "'01-'02", "'02-'03",
                    "'03-'04", "'04-'05", "'05-'06", "'06-'07",
                    "'07-'08", "'08-'09", "'09-'10", "'10-'11",
                    "'11-'12", "'12-'13", "'13-'14", "'14-'15",
                    "'15-'16", "'16-'17", "'17-'18", "'18-'19",
                    "'19-'20", "'20-'21", "'21-'22", "'22-'23"],
                   rotation=-45,
```

```
                    size=13)
ax.tick_params(bottom=True,
               left=True)
```



Visually depicted, we can see an interesting trend with FGP (Field Goal Percentage) as we do with FGA (Field Goal Attempted) rates as both have an upward trend as we approach the present-day. Even whilst shooting a higher percentage of 3s, NBA players maintaned and adapted by being consistent with their 3-ball at the highest level.

If we delve deeper we can also observe and inflection point in the *14-15* season, where the rate of 3-Pointers made continuously grows upward with a small dip which then returns to being an upward trend in the *22-23* season (thus exceeding and 3-Point made rates in any of the past 30 NBA seasons).

5 2.3 *3-Point Shots by Position*

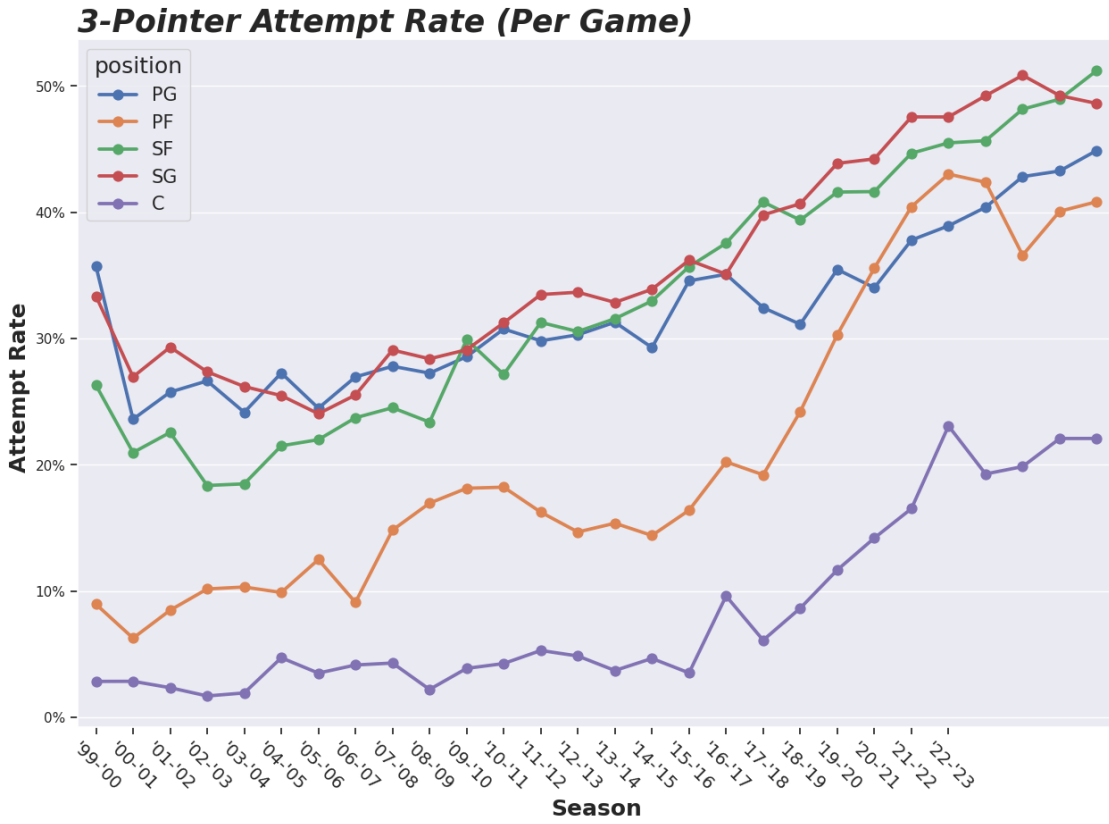
Now we observe this from a top-down Pov looking at all the players. What kind of effect did the 3-Point have on each of the 5 positions (PG, SG, SF, PF, C) in the NBA?

Each positional player obviously having their own unique strengths and weakness when it comes to shooting.

[12]:
```
#Setting up the plot
sns.set(rc={"figure.figsize":(15, 10)})
sns.set_style("darkgrid")
sns.color_palette("tab10")
```

```
#Plotting
ax = sns.pointplot(data=player_shooting,
                   x='season',
                   y='percent_fga_from_x3p_range',
                   errorbar=None,
                   hue='position')
ax.tick_params(bottom=True,
               left=True)


#Configuring titles
ax.axes.set_title("3-Pointer Attempt Rate (Per Game)",
                  fontsize=25,
                  loc='left',
                  fontweight='bold',
                  fontstyle='oblique')
ax.set_xlabel("Season",
              fontsize=18,
              fontweight='bold')
ax.set_ylabel("Attempt Rate ",
              fontsize=18,
              fontweight='bold')


#Formating the y-labels to be percentages instead of decimals
ax.yaxis.set_major_formatter(FuncFormatter(lambda y, _: '{:.0%}'.format(y)))

#Formating the x-labels
ax.set_xticks(range(24))
ax.set_xticklabels(["'99-'00", "'00-'01", "'01-'02", "'02-'03",
                    "'03-'04", "'04-'05", "'05-'06", "'06-'07",
                    "'07-'08", "'08-'09", "'09-'10", "'10-'11",
                    "'11-'12", "'12-'13", "'13-'14", "'14-'15",
                    "'15-'16", "'16-'17", "'17-'18", "'18-'19",
                    "'19-'20", "'20-'21", "'21-'22", "'22-'23"],
                  rotation=-45,
                  size=14)


#Configuring legend
ax.legend(title='position',
          loc='upper left',
          title_fontsize= 18,
          fontsize = 15)
```

[12]: <matplotlib.legend.Legend at 0x7fb544421310>



Initially, at the start of the millenia we see the 3 positions with the most long-range shooting proficiency: Point Guards (PG), Shooting Guards (SG), and Small Forwards (SF) maintain high rates of 3-Pointers attempted ranging from 20-30% per game. The remaining two positions, Centers (C) and Power Forwards (PF) start the millenia shooting an incredibly low rate of 3s with PFs shooting them at a rate of about 10% a game and Cs shooting closer to 2%.

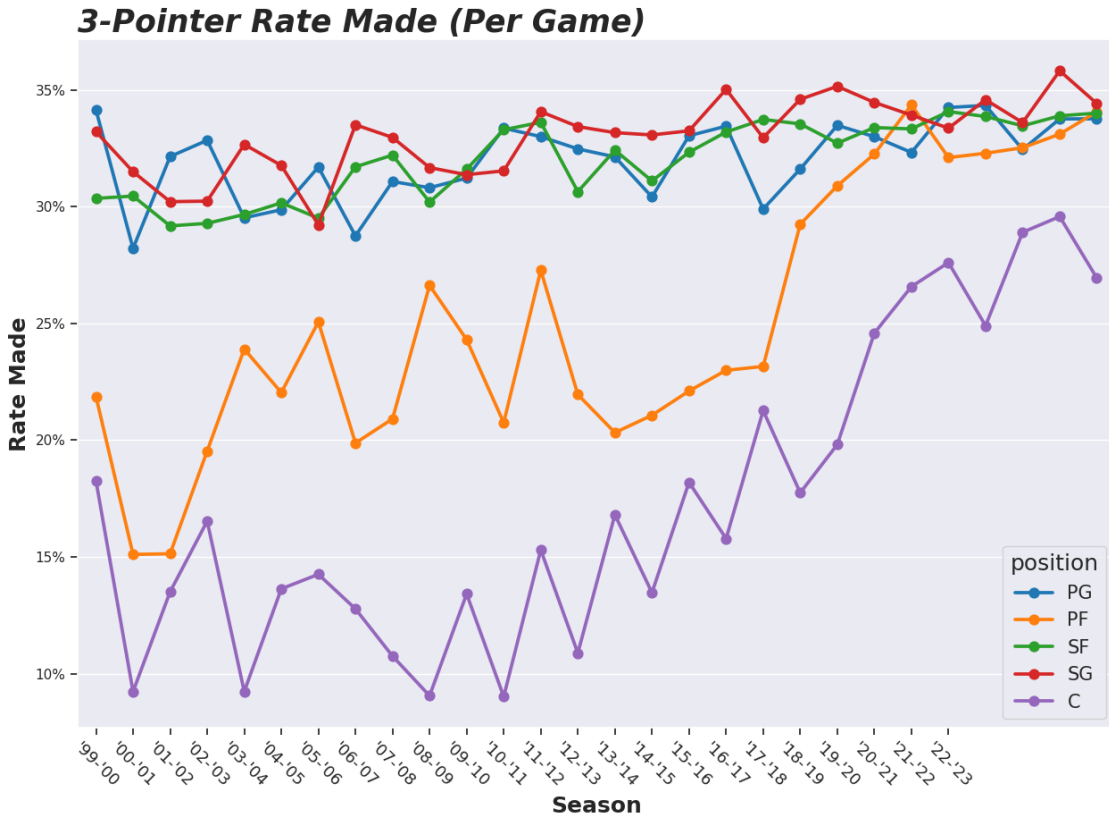
As time progressed however, we see not only a spike in attempts for PGs, SGs, and SFs as one would expect players with the skill set to do so but also from the PFs and Cs who historically are known for dominating the paint to have an increased 3-Point shooting percentange. PFs would go on to shoot 3s at a rate of 40+% per game in some seasons while Cs who would barely attempt a 3-Point shot in the early 2000s would now shoot them at a rate just above 20% per game.

For avid NBA fans this change is undeniable on the court with the last two winner of the NBA MVP award being two of the most prolific Cs in the history of the Game in Joel Embiid and Nikola Jokic. They shot an extensive amount of 3s in each of their MVP-winning seasons and even before using their uncanny shooting abilities paired with their untouchable height.

[13]:
```
#Setting up the plot
fig, ax = plt.subplots(figsize=(15, 10))
sns.set_theme(style="darkgrid", palette="tab10")

#Plotting
```

```
g = sns.pointplot(data=player_shooting,
                  x='season',
                  y='fg_percent_from_x3p_range',
                  errorbar=None,
                  hue='position',
                  ax=ax)
ax.tick_params(bottom=True,
               left=True)

#Configuring titles
ax.axes.set_title('3-Pointer Rate Made (Per Game)',
                  fontsize=25,
                  loc='left',
                  fontweight='bold',
                  fontstyle='oblique')
ax.set_xlabel("Season",
              fontsize=18,
              fontweight='bold')
ax.set_ylabel("Rate Made",
              fontsize=18,
              fontweight='bold')


#Formating the y-labels to be percentages instead of decimals
ax.yaxis.set_major_formatter(FuncFormatter(lambda y, _: '{:.0%}'.format(y)))

#Formating the x-labels
ax.set_xticks(range(24))
ax.set_xticklabels(["'99-'00", "'00-'01", "'01-'02", "'02-'03",
                    "'03-'04", "'04-'05", "'05-'06", "'06-'07",
                    "'07-'08", "'08-'09", "'09-'10", "'10-'11",
                    "'11-'12", "'12-'13", "'13-'14", "'14-'15",
                    "'15-'16", "'16-'17", "'17-'18", "'18-'19",
                    "'19-'20", "'20-'21", "'21-'22", "'22-'23"],
                   rotation=-45,
                   size=13)

#Configuring legend
ax.legend(title='position',
          loc='lower right',
          title_fontsize= 18,
          fontsize = 15)
```

[13]: <matplotlib.legend.Legend at 0x7fb545a745c0>



Looking at the rate at which players were actually making 3s it is not surprising to see PGs, SGs, and SFs shooting at such a high rate but what is more surprising is PFs an Cs quickly catching up with their shorter and quicker counterparts in the 3-Point succession department.

6 2.4 *Team Success*

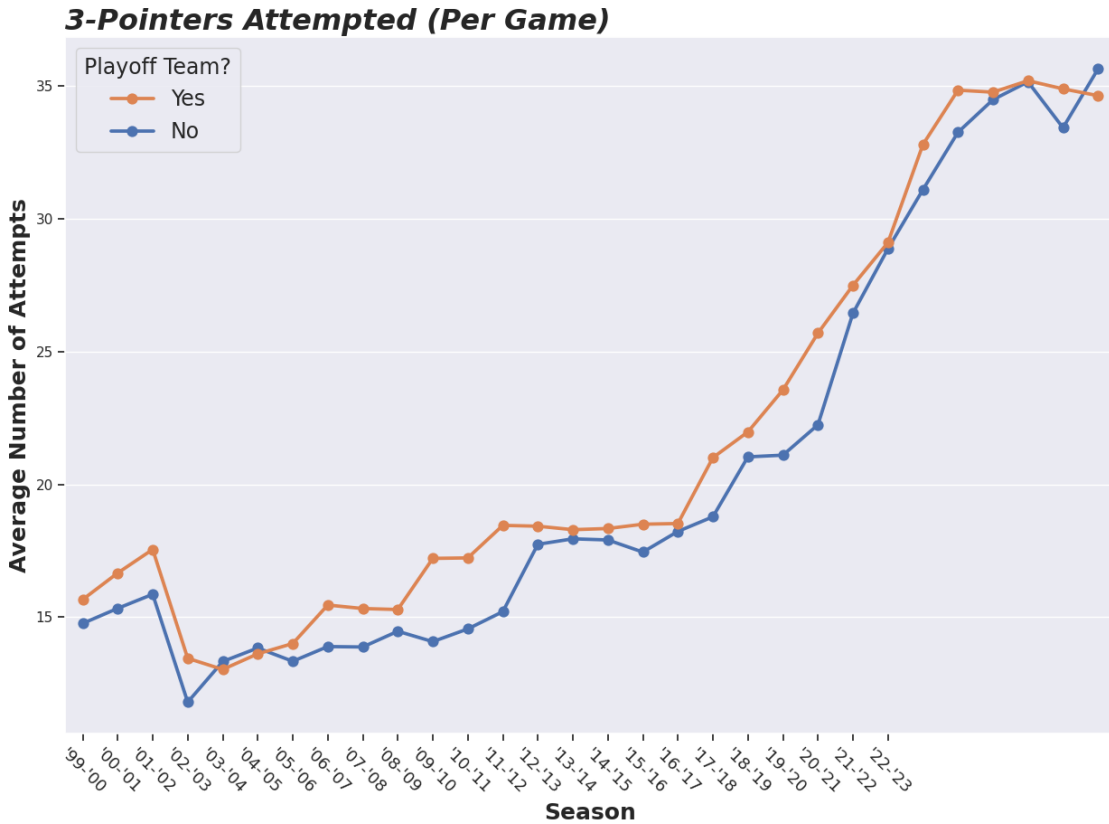
Is there actually any team success coming from these 3-balls?

Using Playoff appearances as a metric of success (making the playoffs signifies a team as being one of the top 8 teams in their respective conference), I have depicted the number of attempts from the 3-point range for a team that made the playoffs in that specific season and those who did not.

[14]:
```
#Setting up the plot
sns.set_theme(style="darkgrid")
sns.set(rc={"figure.figsize":(15, 10)})

#Plotting
ax = sns.pointplot(data=team_totals,
                   x='season',
                   y='x3pa_per_game',
                   hue='playoffs',
                   errorbar=None)
```

```
#Formating the x-labels
ax.set_xticks(range(24))
ax.set_xticklabels(["'99-'00", "'00-'01", "'01-'02", "'02-'03",
                    "'03-'04", "'04-'05", "'05-'06", "'06-'07",
                    "'07-'08", "'08-'09", "'09-'10", "'10-'11",
                    "'11-'12", "'12-'13", "'13-'14", "'14-'15",
                    "'15-'16", "'16-'17", "'17-'18", "'18-'19",
                    "'19-'20", "'20-'21", "'21-'22", "'22-'23"],
                  rotation=-45, size=13)
ax.tick_params(bottom=True,
               left=True)

#Configuring titles
ax.axes.set_title('3-Pointers Attempted (Per Game)',
                  fontsize=23,
                  loc='left',
                  fontweight='bold',
                  fontstyle='oblique')
ax.set_xlabel("Season",
              fontsize=18,
              fontweight='bold')
ax.set_ylabel("Average Number of Attempts",
              fontsize=18,
              fontweight='bold')

#Configuring legend
handles, labels = ax.get_legend_handles_labels()
ax.legend(list(reversed(handles)),
          list(reversed(["No", "Yes"])),
          title='Playoff Team?',
          loc='upper left',
          title_fontsize= 17,
          fontsize = 17)
```

[14]: <matplotlib.legend.Legend at 0x7fb53c7d2de0>



From this graph, we can see that teams that consistently made the playoffs continually attempted more 3s than their non playoff counterparts since the beginning of the early 2000s (albeit by a small margin).

7 3. *Shot Mapping*

7.1 3.1 Shot Frequency

To get a thorough understanding of what was actually happening on the court we can examine the shot location data which gives us the ability to map onto a plane all the shots attempted during an entire season.

> **Footnote:** Due to a majority of basketball shots happening directly underneath or next to the rim, a maximum cutoff for shots was established when visualizing the entire court so hexbins around the rim would not drown out other hexbins. The maximum cutoff that was chosen was 400 shots.

[15]:
```
#Drawing basketball court
def draw_court(ax=None, color='white', lw=2, outer_lines=False):
    #If an axes object isn't provided to plot onto, just get current one
    if ax is None:
        ax = plt.gca()
```

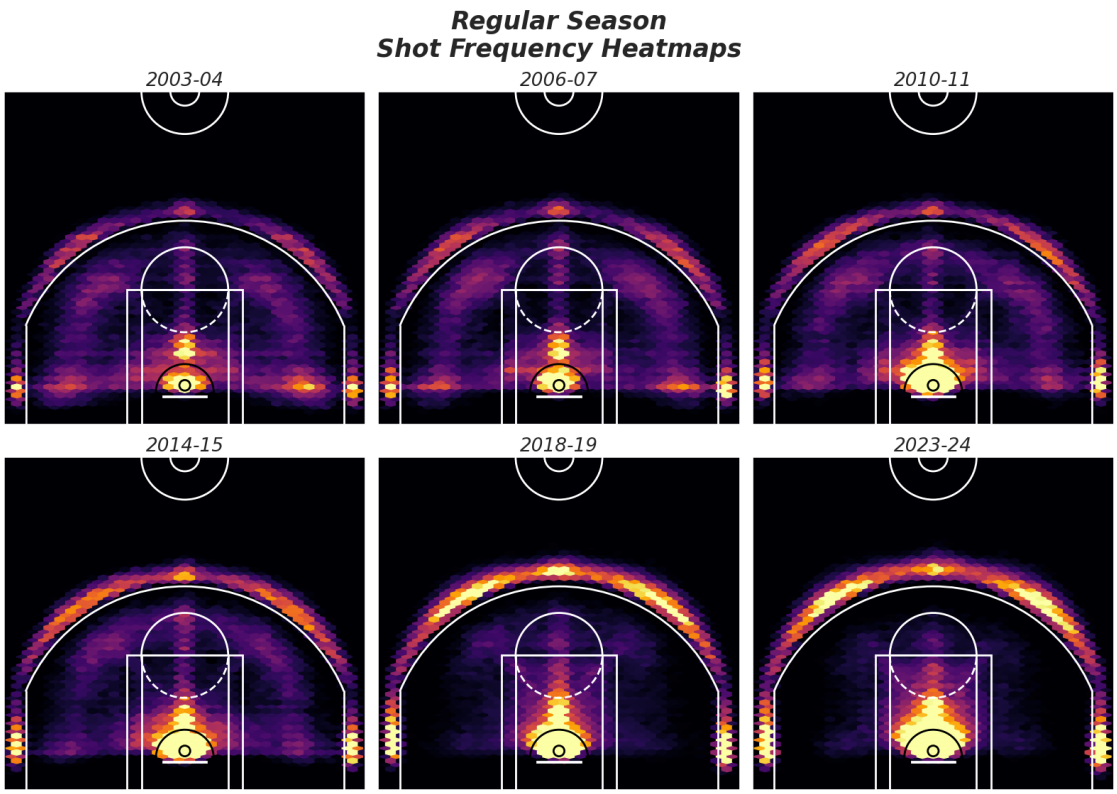
```
#Create the various parts of an NBA basketball court

#Create the basketball hoop
hoop = Circle((0, 55), radius=7.5, linewidth=lw, color="black", fill=False)

#Create backboard
backboard = Rectangle((-30, 40), 60, -1, linewidth=lw, color=color)

#The paint
outer_box = Rectangle((-80, 0), 160, 190, linewidth=lw, color=color,
                      fill=False)
#Create the inner box of the paint, widt=12ft, height=19ft
inner_box = Rectangle((-60, 0), 120, 190, linewidth=lw, color=color,
                      fill=False)

#Create free throw top arc
top_free_throw = Arc((0, 190), 120, 120, theta1=0, theta2=180,
                     linewidth=lw, color=color, fill=False)
#Create free throw bottom arc
bottom_free_throw = Arc((0, 190), 120, 120, theta1=180, theta2=0,
                        linewidth=lw, color=color, linestyle='dashed')
#Restricted Zone, it is an arc with 4ft radius from center of the hoop
restricted = Arc((0, 45), 80, 80, theta1=0, theta2=180, linewidth=lw,
                 color="black")

#Three point line
#Create the side 3pt lines, they are 14ft long before they begin to arc
corner_three_a = Rectangle((-220, 0), 0, 140, linewidth=lw,
                           color=color)
corner_three_b = Rectangle((220, 0), 0, 140, linewidth=lw, color=color)
#3pt arc - center of arc will be the hoop, arc is 23'9" away from hoop
#I just played around with the theta values until they lined up with the␣
↪threes
three_arc = Arc((0, 50), 475, 475, theta1=22, theta2=158, linewidth=lw,
                color=color)

#Center Court
center_outer_arc = Arc((0, 470), 120, 120, theta1=180, theta2=0,
                       linewidth=lw, color=color)
center_inner_arc = Arc((0, 470), 40, 40, theta1=180, theta2=0,
                       linewidth=lw, color=color)

#List of the court elements to be plotted onto the axes
court_elements = [hoop, backboard, outer_box, inner_box, top_free_throw,
                  bottom_free_throw, restricted, corner_three_a,
                  corner_three_b, three_arc, center_outer_arc,
```

```
                        center_inner_arc]

    if outer_lines:
        #Draw the half court line, baseline and side out bound lines
        outer_lines = Rectangle((-250, 0), 500, 470, linewidth=lw,
                                color=color, fill=False)
        court_elements.append(outer_lines)

    #Add the court elements onto the axes
    for element in court_elements:
        ax.add_patch(element)

    return ax
```

[16]:
```
#Setting up plot
fig, ax = plt.subplots(nrows=2, ncols=3, figsize=(16,11.4),  sharex=True,␣
 ↪sharey=True)
plt.xlim(-250, 250)
plt.ylim(0, 470)


#Ploting each court
#We use larger values than the axis limits to ensure that hexagons go all the␣
 ↪way to the boundaries of our draw court
ax1= ax[0,0].hexbin(shots_04['LOC_X'],
           shots_04['LOC_Y'],
           gridsize=(40, 40),
           extent=(-300, 300, 0, 470),
           cmap='inferno',
           vmin=20,
           vmax=450)

ax2 = ax[0,1].hexbin(shots_07['LOC_X'],
           shots_07['LOC_Y'],
           gridsize=(40, 40),
           extent=(-300, 300, 0, 470),
           cmap='inferno',
           vmin=20,
           vmax=450)

ax3 = ax[0,2].hexbin(shots_11['LOC_X'],
           shots_11['LOC_Y'],
           gridsize=(40, 40),
           extent=(-300, 300, 0, 470),
           cmap='inferno',
           vmin=20,
           vmax=450)
```

```
ax4 = ax[1,0].hexbin(shots_15['LOC_X'],
           shots_15['LOC_Y'],
           gridsize=(40, 40),
           extent=(-300, 300, 0, 470),
           cmap='inferno',
           vmin=20,
           vmax=450)

ax5 = ax[1,1].hexbin(shots_19['LOC_X'],
           shots_19['LOC_Y'],
           gridsize=(40, 40),
           extent=(-300, 300, 0, 470),
           cmap='inferno',
           vmin=20,
           vmax=450)

ax6 = ax[1,2].hexbin(shots_23["LOC_X"],
           shots_23["LOC_Y"],
           gridsize=(40, 40),
           extent=(-300, 300, 0, 470),
           cmap="inferno",
           vmin=20,
           vmax=450)

#Add labels to each court image
ax[0, 0].set_title('2003-04', fontsize=19, fontweight='medium',␣
 ↪fontstyle='oblique')
ax[0, 1].set_title('2006-07', fontsize=19, fontweight='medium',␣
 ↪fontstyle='oblique')
ax[0, 2].set_title('2010-11', fontsize=19, fontweight='medium',␣
 ↪fontstyle='oblique')
ax[1, 0].set_title('2014-15', fontsize=19, fontweight='medium',␣
 ↪fontstyle='oblique')
ax[1, 1].set_title('2018-19', fontsize=19, fontweight='medium',␣
 ↪fontstyle='oblique')
ax[1, 2].set_title('2022-23', fontsize=19, fontweight='medium',␣
 ↪fontstyle='oblique')

#Costimizing axis ticks and adding courts
for row in range(2):
    for col in range(3):
        ax[row, col].get_xaxis().set_visible(False)
        ax[row, col].get_yaxis().set_visible(False)
        draw_court(ax[row,col])
```

```
fig.suptitle("Regular Season\nShot Frequency Heatmaps", fontsize=25,␣
↪fontweight='bold', fontstyle='oblique')
fig.tight_layout()
plt.show()
```



From charts of the *2003-2004* season to that of *2022-2023* season we see a gradual movement away from the mid-range shot (long 2-pointer) to an increase in 3-pointers as the years went on. Alongside the increase in 3s, there was also an increase in shots concentrating around the ring pointing to a polarity in shot selection for players; in their minds, it's either I shot a 3-Pointer or I take a more efficient/easier shot around the rim.

The death of the midrange (long 2-pointer) shot is clear to see here with the heat map showing shot regions from the *2003-04* season being brighter in the space between the 3-Point line and the key (aka the paint). That brightness/volume shot volume then slowly starts to migrate further and further.

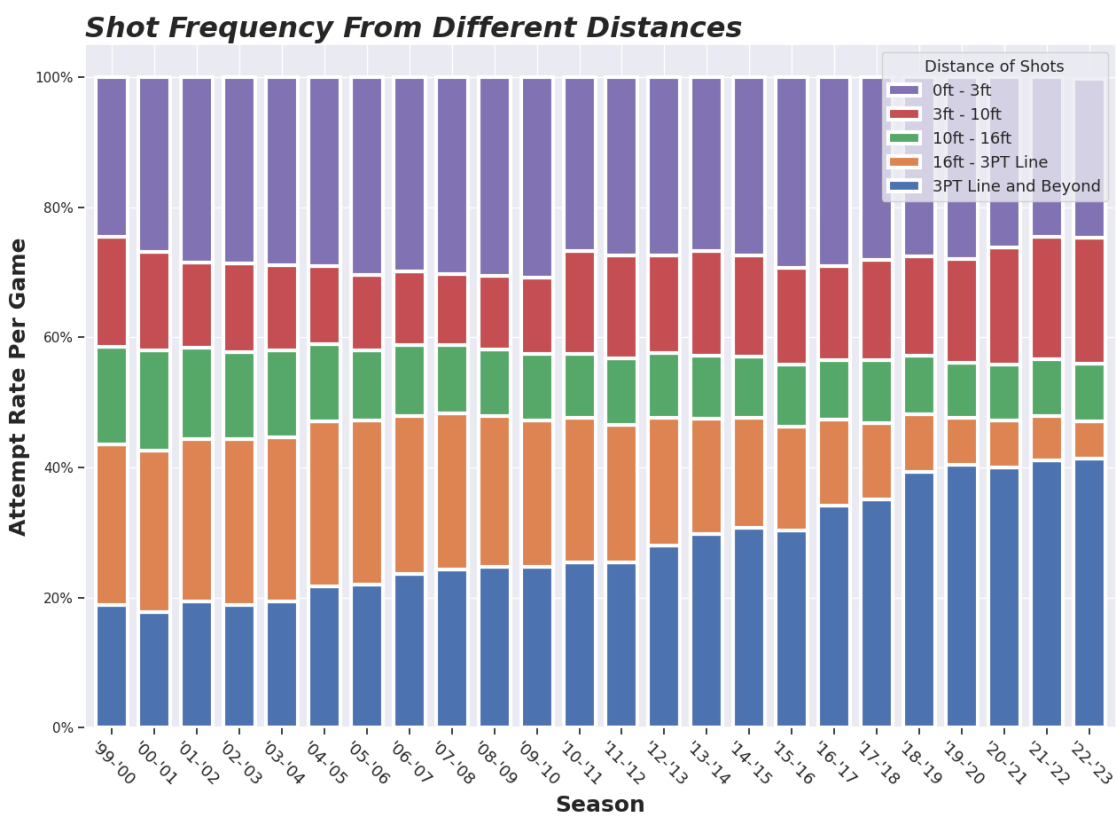
8 3.2 *Shot Distance Distribution*

What interesting information can we find if we visualize the percentage of shots taken from a specific shot distance?

[17]:
```
#Setting up the plot
sns.set(rc={"figure.figsize":(15, 10)})
sns.color_palette("tab10")
sns.set(font_scale=1)

attempted = player_shooting.groupby('season')\
            [['percent_fga_from_x3p_range',␣
 ↪'percent_fga_from_x16_3p_range','percent_fga_from_x10_16_range', ␣
 ↪'percent_fga_from_x3_10_range', 'percent_fga_from_x0_3_range']]\
            .mean()\
            .reset_index()

# Filter attempted DataFrame to match the number of labels
attempted = attempted.loc[(attempted['season'] > 1999) & (attempted['season'] <␣
 ↪2024)]

#Plotting
ax = attempted.set_index('season').plot(kind='bar',
                                        stacked=True,
                                        width = 0.75,
                                        edgecolor = "white",
                                        linewidth=3)

#Formating the ylabels to be percentages instead of decimals
ax.yaxis.set_major_formatter(FuncFormatter(lambda y, _: '{:.0%}'.format(y)))

#Formating the xlabels
ax.set_xticks(range(len(attempted.set_index('season'))))
ax.set_xticklabels(["'99-'00", "'00-'01", "'01-'02", "'02-'03",
                    "'03-'04", "'04-'05", "'05-'06", "'06-'07",
                    "'07-'08", "'08-'09", "'09-'10", "'10-'11",
                    "'11-'12", "'12-'13", "'13-'14", "'14-'15",
                    "'15-'16", "'16-'17", "'17-'18", "'18-'19",
                    "'19-'20", "'20-'21", "'21-'22", "'22-'23"],
                   rotation=-45,
                   size=13)
ax.tick_params(bottom=True,
               left=True)

#Configuring titles
ax.axes.set_title('Shot Frequency From Different Distances',
                  fontsize=23,
                  loc='left',
                  fontweight='bold',
                  fontstyle='oblique')
ax.set_xlabel("Season",
              fontsize=18,
```

```
                fontweight='bold')
ax.set_ylabel("Attempt Rate Per Game",
                fontsize=18,
                fontweight='bold')


#Configuring legend
handles, labels = ax.get_legend_handles_labels()
ax.legend(list(reversed(handles)),
          list(reversed(["3PT Line and Beyond", "16ft - 3PT Line", "10ft -␣
 ↪16ft", "3ft - 10ft", "0ft - 3ft"])),
          title='Distance of Shots',
          loc='upper right',
          title_fontsize= 13,
          fontsize = 13)
```

[17]: <matplotlib.legend.Legend at 0x7fb53c61b650>



Looking into the percentage of shots attempted from certain distances on the court one can see most shot rates from certain distances staying around a similar percentage range while one in particular almost being gone completely. The "16ft to 3-Point line shot (24ft)" is almost going extinct with the boom of the 3-Point shot eating away at shots that would be taken at that distance historically.

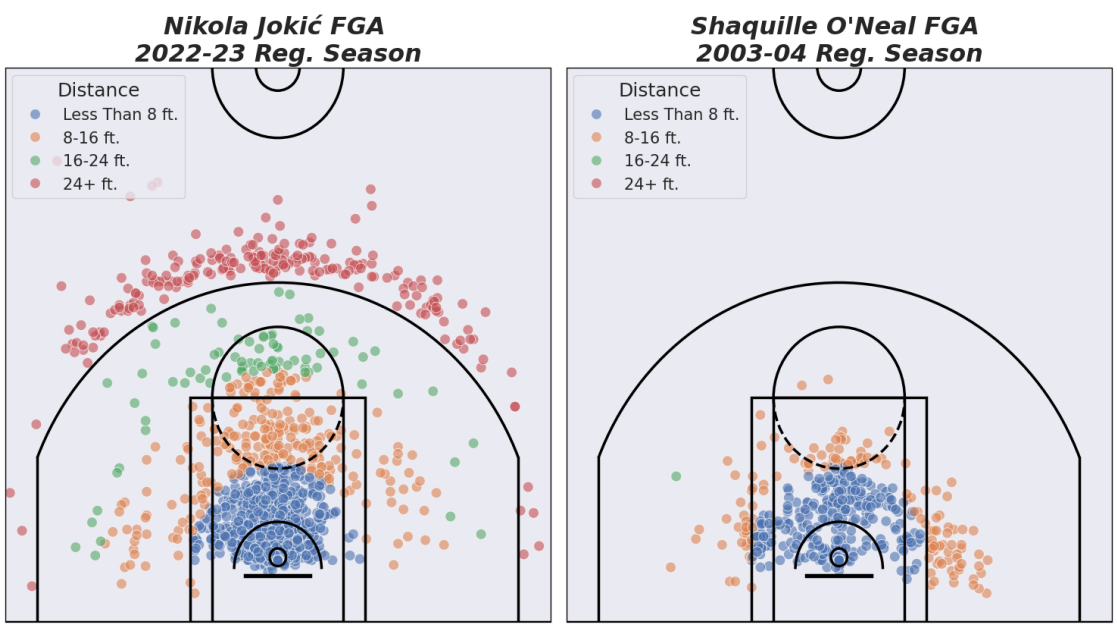
However, the mid-range shot (long 2-pointer) isn't necessarily dead as stars like Kevin Durant, Demar Derozan, and Devin Booker are still known for incorporating such a shot into their shot portfolio.

9 3.3 *Jokic vs Shaq*

The difference in shot selection for players is made even more evident by looking at the shot selection of the most dominant *Centers* from their respective eras, Shaquille O'Neal and Nikola Jokic.

[19]:
```
#Creating our two new dataframes
shaq_04 = shots_04.query('PLAYER_ID == 406 & ZONE_RANGE != "Back Court Shot"')
jokic_23 = shots_23.query('PLAYER_NAME == "Nikola Jokic" & ZONE_RANGE != "Back␣
↪Court Shot"')


#Setting up subplots
fig, ax = plt.subplots(ncols=2, figsize=(15,8.4),  sharex=True, sharey=True)
sns.set_theme(style="dark")


#Ploting our points
sns.scatterplot(data = shaq_04,
                x='LOC_X',
                y='LOC_Y',
                hue='ZONE_RANGE',
                hue_order=['Less Than 8 ft.', '8-16 ft.', '16-24 ft.', '24+ ft.
↪'],
                alpha=0.6,
                s=100,
                ax=ax[1])

sns.scatterplot(data = jokic_23,
                x='LOC_X',
                y='LOC_Y',
                hue='ZONE_RANGE',
                hue_order=['Less Than 8 ft.', '8-16 ft.', '16-24 ft.', '24+ ft.
↪'],
                alpha=0.6,
                s=100,
                ax=ax[0])


#Add labels to each court image
ax[1].set_title("Shaquille O'Neal FGA \n2003-04 Reg. Season",
                fontsize=23,
                fontweight='bold',
                fontstyle='oblique')
```

```
ax[0].set_title("Nikola Jokić FGA \n2022-23 Reg. Season",
                fontsize=23,
                fontweight='bold',
                fontstyle='oblique')

#Legends
ax[1].legend(title='Distance',
             loc='upper left',
             title_fontsize= 18,
             fontsize = 15)

ax[0].legend(title='Distance',
             loc='upper left',
             title_fontsize = 18,
             fontsize = 15)

#Costimizing axes
for col in range(2):
    ax[col].get_xaxis().set_visible(False)
    ax[col].get_yaxis().set_visible(False)
    draw_court(ax[col], color='black', outer_lines=True, lw=2.5)

#Dsiplaying plots
fig.tight_layout()
plt.xlim(-250, 251)
plt.ylim(-1, 470)
fig.show()
```



The singular green dot in Shaq's shot chart highlights the drastic change in Center's shot selection from the early 2000s to today

10 4. *Building a Scouting Report (Past vs Present)*

Let us take a more granular look at how shooting habits have changed over time by comparing two seasons of Lebron James' career and utilizing machine learning models. Why pick Lebron James for this evaluation? He provides us with a very unique look into a player that still plays at the relatively same pace and efficiency today as he did in the early 2000s when he started his NBA career. Still averaging 25+ points per game in his 21st season he's found ways to evolve his playstyle with the times making him a perfect candidate for our comparison.

11 4.1 Data Preprocessing

[20]:
```
#Creating our two new dataframes for our ML models
lebron_04 = shots_04.query('PLAYER_NAME == "LeBron James" & ZONE_RANGE != "Back␣
↪Court Shot"')

#Getting DataFrame ready for preprocessing by removing remaining columns we␣
↪don't need
lebron04_ml = lebron_04.drop(['SEASON_2', 'TEAM_NAME', 'PLAYER_ID',␣
↪'PLAYER_NAME', 'EVENT_TYPE',
                              'SHOT_TYPE', 'ZONE_NAME', 'ZONE_ABB',␣
↪'ZONE_RANGE', 'BASIC_ZONE',
                              'ACTION_TYPE','QUARTER', 'MINS_LEFT',␣
↪'SECS_LEFT'],
                             axis=1,
                             inplace=False)

#Features
X_04 = lebron04_ml.iloc[:, :-1].values
#Labels
y_04 = lebron04_ml.iloc[:, -1].values

#Splitting training set into Train and Test sets
X_train_04, X_test_04, y_train_04, y_test_04 = train_test_split(X_04, y_04,␣
↪test_size = 0.25, random_state = 42)

#Displaying our new dataset for model training
print("2003-2004 Shooting Dataset")
lebron04_ml
```

```
2003-2004 Shooting Dataset
```

[20]:

	LOC_X	LOC_Y	SHOT_DISTANCE	SHOT_MADE
258	-0.0	52.5	0	1
307	-0.0	52.5	0	0
431	210.0	203.5	25	0
570	-0.0	52.5	0	1
632	-0.0	52.5	0	1
…	…	…	…	…
189090	-54.0	74.5	5	1
189149	118.0	144.5	14	0
189170	-163.0	57.5	16	1
189202	131.0	54.5	13	1
189240	-158.0	52.5	15	1

[1489 rows x 4 columns]

[22]:

```
#Creating our two new dataframes for our ML models
lebron_23 = shots_23.query('PLAYER_NAME == "LeBron James" & ZONE_RANGE != "Back␣
↪Court Shot"')


#Getting DataFrame ready for preprocessing by removing remaining columns we␣
↪don't need
lebron23_ml = lebron_23.drop(['SEASON_2', 'TEAM_NAME', 'PLAYER_ID',␣
↪'PLAYER_NAME', 'EVENT_TYPE',
                              'SHOT_TYPE', 'ZONE_NAME', 'ZONE_ABB',␣
↪'ZONE_RANGE', 'BASIC_ZONE',
                              'ACTION_TYPE', 'QUARTER', 'MINS_LEFT',␣
↪'SECS_LEFT'],
                             axis=1,
                             inplace=False)


#Features
X_23 = lebron23_ml.iloc[:, :-1].values
#Labels
y_23 = lebron23_ml.iloc[:, -1].values

#Splitting training set into Train and Test sets
X_train_23, X_test_23, y_train_23, y_test_23 = train_test_split(X_23, y_23,␣
↪test_size = 0.25, random_state = 42)

#Displaying our new dataset for model training
print("2023 Shooting Dataset")
lebron23_ml
```

2023 Shooting Dataset

[22]:

	LOC_X	LOC_Y	SHOT_DISTANCE	SHOT_MADE
116069	47.0	224.5	17	0
116070	-2.0	57.5	0	1
116072	119.0	84.5	12	0
116074	-1.0	60.5	0	1
116082	82.0	141.5	12	0
…	…	…	…	…
123189	1.0	143.5	9	1
123200	-10.0	54.5	1	1
123205	36.0	53.5	3	1
123211	-23.0	321.5	26	1
123213	12.0	48.5	1	0

[1268 rows x 4 columns]

12 4.2 *Model Selection and Hyperparameter Tuning*

Let's start the process of building and comparing different classification models with default parameters and then applying more complex tools to make our models more accurate.

Important note that both datasets for each season will be tested on models separately to ensure that the appropriate model is being chosen correctly for each dataset.

12.0.1 *2003-2004 Season Dataset: Multi-Model Analysis*

Training and Performce Metrics w/ Default Parameters

[23]:
```
######################################## MODEL PERFORMANCE 2004␣
↪##################################

#Creating a dictionary of the models we'll be using
class_models = {"Logisitic Regression": LogisticRegression(),
                "Random Forest Classifier": RandomForestClassifier(),
                "K-nearest neighbors": KNeighborsClassifier(),
                "Decision Tree Classifier": DecisionTreeClassifier(),
                "Gradient Boosting Classifier": GradientBoostingClassifier(),
                "Support Vector Machine": SVC()}

# Train and evaluate each classifier
results = {'Model': [], 'Accuracy': [], 'Precision': [], 'Recall': [], 'F1␣
↪Score': []}

for clf_name, clf in class_models.items():
    # Train the classifier
    clf.fit(X_train_04, y_train_04)

    # Make predictions on the test set
```

```
    y_pred_04 = clf.predict(X_test_04)

    # Evaluate the classifier
    accuracy = round((accuracy_score(y_test_04, y_pred_04) * 100), 4)
    precision = round((precision_score(y_test_04, y_pred_04) * 100), 4)
    recall = round((recall_score(y_test_04, y_pred_04) * 100), 4)
    f1 = round((f1_score(y_test_04, y_pred_04) * 100), 4)

    # Save results
    results['Model'].append(clf_name)
    results['Accuracy'].append(accuracy)
    results['Precision'].append(precision)
    results['Recall'].append(recall)
    results['F1 Score'].append(f1)

# Create a DataFrame to display the results
results_df = pd.DataFrame(results)
results_df.set_index('Model', inplace=True)
results_df.sort_values(by='Accuracy', ascending=False).style.
 ↪background_gradient(cmap='Blues')
```

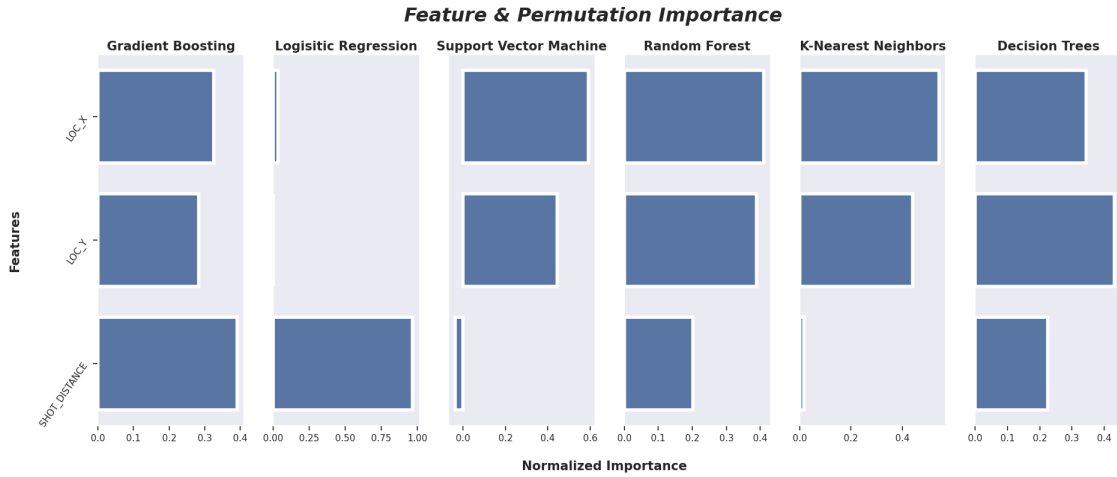
[23]: <pandas.io.formats.style.Styler at 0x7fb544164aa0>

Feature Importance w/ Default Parameters

[24]:
```
#Convert X_train to a DataFrame
X_train_df_04 = pd.DataFrame(X_train_04)

#Organize classifiers
models = [GradientBoostingClassifier(),
          LogisticRegression(),
          SVC(),
          RandomForestClassifier(),
          KNeighborsClassifier(),
          DecisionTreeClassifier()]

model_names = ["Gradient Boosting",
               "Logisitic Regression",
               "Support Vector Machine",
               "Random Forest",
               "K-Nearest Neighbors",
               "Decision Trees"]

#Creating subplots
fig, axes = plt.subplots(nrows=1, ncols=len(models), figsize=(22, 8),␣
 ↪sharey='row')
sns.set_theme(style="darkgrid")
```

```
#Plot importance for each model
for i, (model, name) in enumerate(zip(models, model_names)):
    model.fit(X_train_04, y_train_04)

    #Feature importance for tree-based models (Random Forest, Decision Trees,␣
↪Gradient Boosting)
    if hasattr(model, 'feature_importances_'):
        importance = model.feature_importances_

    #Feature importance for linear models (Logistic Regression)
    elif hasattr(model, 'coef_'):
        importance = np.abs(model.coef_[0])

    #Permutation importance for models without feature importance (KNN and SVM)
    else:
        result = permutation_importance(model, X_train_04, y_train_04,␣
↪n_repeats=10, random_state=42)
        importance = result.importances_mean

    #Normalize the importance values for better comparison
    if importance is not None:

        importance = importance / np.sum(importance)

        #Plot the bar chart
        sns.barplot(x=importance, y=X_train_df_04.columns, ax=axes[i],␣
↪orient='h',
                    width = 0.75, edgecolor = 'white', linewidth=4)

        #Customize titles
        axes[i].set_title(f'{name}', fontsize=15, fontweight='bold')

        #Customize x-axis
        axes[2].set_xlabel('\nNormalized Importance', fontsize=15,␣
↪fontweight='bold', ha='left')
        axes[0].tick_params(left=True)

        #Customize y-axis
        axes[i].set_yticks(np.arange(len(importance)))
        axes[0].set_yticklabels(('LOC_X', 'LOC_Y', 'SHOT_DISTANCE'),␣
↪rotation=55, fontsize=11)
        axes[0].set_ylabel('Features\n', fontsize=15, fontweight='bold')
        axes[i].tick_params(bottom=True)

#Show plot
```

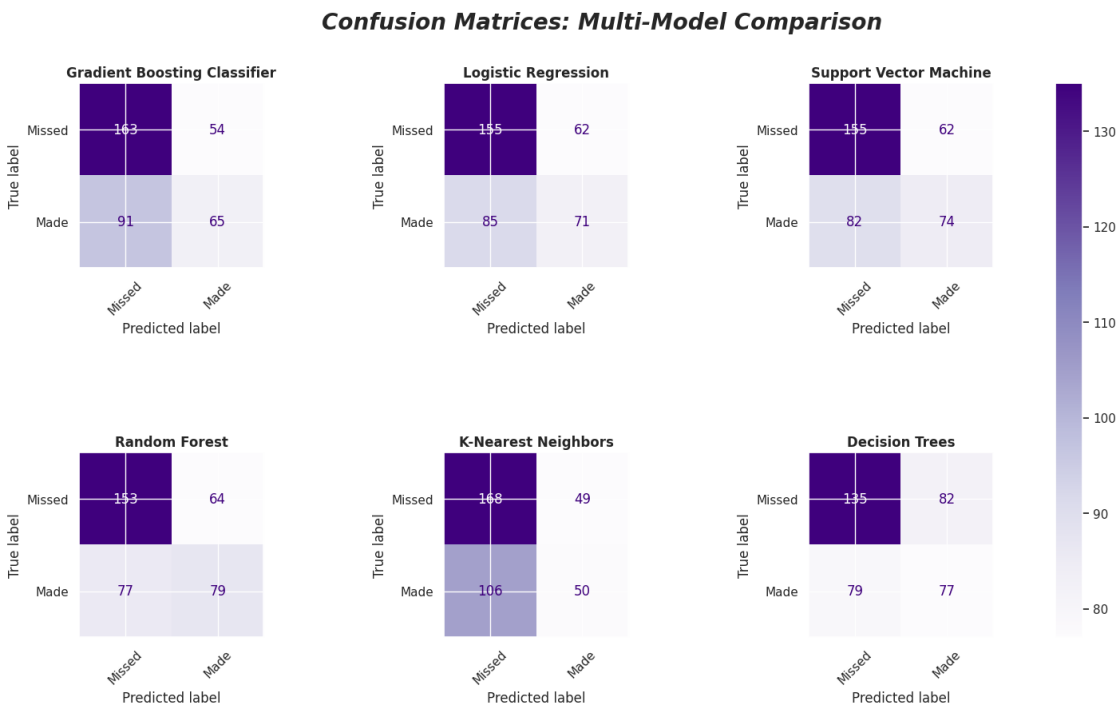
```
fig.suptitle("Feature & Permutation Importance\n", fontsize=23,␣
 ↪fontweight='bold', fontstyle='oblique')
plt.show()
```



Confusion Matrices w/ Default Parameters

[25]:
```
################################ Confusion Matrix 2004␣
 ↪#######################################
#Creating subplots
fig, axes = plt.subplots(2, 3, figsize=(20, 9))
class_models = {"Gradient Boosting Classifier": GradientBoostingClassifier(),
                "Logistic Regression": LogisticRegression(),
                "Support Vector Machine": SVC(),
                "Random Forest": RandomForestClassifier(),
                "K-Nearest Neighbors": KNeighborsClassifier(),
                "Decision Trees": DecisionTreeClassifier()}


#Creating matrices
for i, (key, model) in enumerate(class_models.items()):
    y_pred_04 = model.fit(X_train_04, y_train_04).predict(X_test_04)
    disp = ConfusionMatrixDisplay(confusion_matrix=confusion_matrix(y_test_04,␣
 ↪y_pred_04),
                                  display_labels=['Missed', 'Made'])
    disp.plot(ax=axes[i // 3, i % 3], xticks_rotation=45, cmap='Purples')
    disp.ax_.set_title(key, fontsize=12, fontweight='bold')
    disp.im_.colorbar.remove()
    disp.ax_.set_xlabel('Predicted label')
    if i // 3 != 1:
        disp.ax_.set_ylabel('True label')
```

```
#Customizing plot
plt.subplots_adjust(wspace=0.40, hspace=1)
fig.suptitle("Confusion Matrices: Multi-Model Comparison", fontsize=20,
             fontweight='bold', fontstyle='oblique', ha='center')
fig.colorbar(disp.im_, ax=axes)
fig.set_tight_layout(False)
fig.show()
```



Applying k-Fold Cross Validation

[26]:
```
############################## Applying k-Fold Cross Validation 2004␣
↪####################

#Creating a dictionary of the models we'll be using
class_models = {"Logisitic Regression": LogisticRegression(),
                "Random Forest Classifier": RandomForestClassifier(),
                "K-nearest neighbors": KNeighborsClassifier(),
                "Decision Tree Classifier": DecisionTreeClassifier(),
                "Gradient Boosting Classifier": GradientBoostingClassifier(),
                "Support Vector Machine": SVC()}

# Train and evaluate each classifier
results = {'Model': [], 'Average Accuracy': [], 'Standard Deviation': []}
```

```
for clf_name, clf in class_models.items():
    # Train the classifier
    clf.fit(X_train_04, y_train_04)

    # Evaluate the classifier
    accuracies = cross_val_score(estimator = clf, X = X_train_04, y =␣
 ↪y_train_04, cv = 10)
    accuracy = round((accuracies.mean()*100), 4)
    sd = round((accuracies.std()*100), 4)

    # Save results
    results['Model'].append(clf_name)
    results['Average Accuracy'].append(accuracy)
    results['Standard Deviation'].append(sd)

# Create a DataFrame to display the results
results_df = pd.DataFrame(results)
results_df.set_index('Model', inplace=True)
results_df.sort_values(by='Average Accuracy', ascending=False).style.
 ↪background_gradient(cmap='Greens')
```

[26]: <pandas.io.formats.style.Styler at 0x7fb5443e2f30>

Hyperparameter Tuning for our Best performing model (Gradient Boosting)

[27]:
```
############################## Using GridSearchCV ####################

#Define the parameter grid for GridSearchCV
param_grid = {'n_estimators': [50, 100, 200],
              'learning_rate': [0.01, 0.1, 0.2, 0.3, 0.4, 0.5],
              'max_depth': [1, 3, 5, 7]}

#Initialize the Gradient Boosting model
gb_model = GradientBoostingClassifier()

#Initialize GridSearchCV
grid_search = GridSearchCV(estimator=gb_model, param_grid=param_grid, cv=10,␣
 ↪scoring='accuracy', n_jobs=-1)

#Fit the model to the training data using GridSearchCV
grid_search.fit(X_train_04, y_train_04)

#Get the best parameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

#Make predictions on the test set using the best model
```

```
y_pred_best_04 = best_model.predict(X_test_04)

#Evaluate the best model
accuracy_best = accuracy_score(y_test_04, y_pred_best_04)

#Print results
print("Best Parameters (Grid Search):", best_params)
print(f"Best Model Accuracy (Grid Search): {accuracy_best}")
```

```
Best Parameters (Grid Search): {'learning_rate': 0.01, 'max_depth': 1,
'n_estimators': 50}
Best Model Accuracy (Grid Search): 0.6327077747989276
```

12.0.2 *2022-2023 Season Dataset: Model Analysis*

Training and Performance Metrics w/Default Parameters

[28]:
```
######################################## MODEL PERFORMANCE 2023␣
↪##################################
#Creating a dictionary of the models we'll be using
class_models = {"Logisitic Regression": LogisticRegression(),
                "Random Forest Classifier": RandomForestClassifier(),
                "K-nearest neighbors": KNeighborsClassifier(),
                "Decision Tree Classifier": DecisionTreeClassifier(),
                "Gradient Boosting Classifier": GradientBoostingClassifier(),
                "Support Vector Machine": SVC()}

# Train and evaluate each classifier
results = {'Model': [], 'Accuracy': [], 'Precision': [], 'Recall': [], 'F1␣
↪Score': []}

for clf_name, clf in class_models.items():
    # Train the classifier
    clf.fit(X_train_23, y_train_23)

    # Make predictions on the test set
    y_pred_23 = clf.predict(X_test_23)

    # Evaluate the classifier
    accuracy = round((accuracy_score(y_test_23, y_pred_23) * 100), 4)
    precision = round((precision_score(y_test_23, y_pred_23) * 100), 4)
    recall = round((recall_score(y_test_23, y_pred_23) * 100), 4)
    f1 = round((f1_score(y_test_23, y_pred_23) * 100), 4)

    # Save results
    results['Model'].append(clf_name)
    results['Accuracy'].append(accuracy)
```

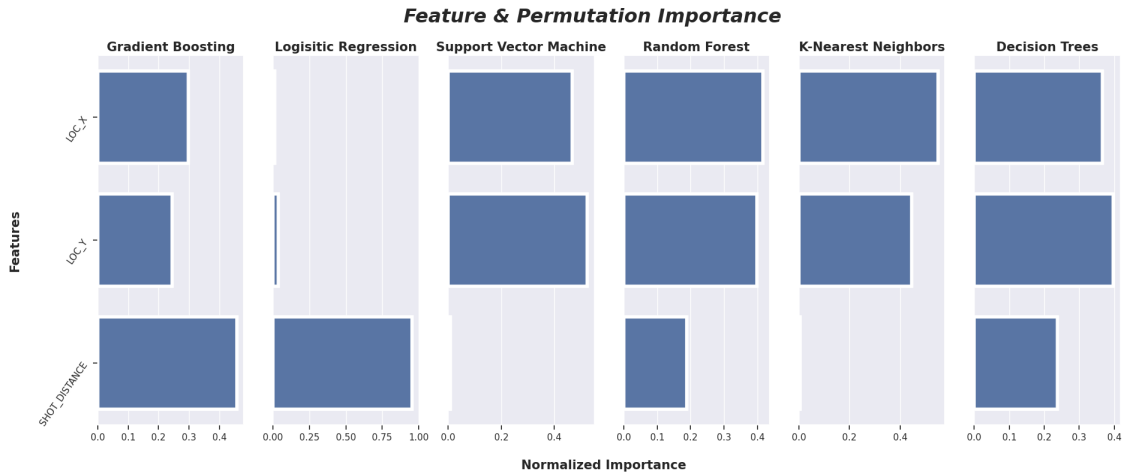
```
    results['Precision'].append(precision)
    results['Recall'].append(recall)
    results['F1 Score'].append(f1)

# Create a DataFrame to display the results
results_df = pd.DataFrame(results)
results_df.set_index('Model', inplace=True)
results_df.sort_values(by='Accuracy', ascending=False).style.
 ↪background_gradient(cmap='Blues')
```

[28]: <pandas.io.formats.style.Styler at 0x7fb5442b9af0>

Feature Importance w/Default Parameters

[29]:
```
#Convert X_train to a DataFrame
X_train_df_23 = pd.DataFrame(X_train_23)

#Organize classifiers
models = [GradientBoostingClassifier(),
          LogisticRegression(),
          SVC(),
          RandomForestClassifier(),
          KNeighborsClassifier(),
          DecisionTreeClassifier()]

model_names = ["Gradient Boosting",
               "Logisitic Regression",
               "Support Vector Machine",
               "Random Forest",
               "K-Nearest Neighbors",
               "Decision Trees"]

#Creating subplots
fig, axes = plt.subplots(nrows=1, ncols=len(models), figsize=(22, 8),␣
 ↪sharey='row')
sns.set_theme(style="darkgrid")

#Plot importance for each model
for i, (model, name) in enumerate(zip(models, model_names)):
    model.fit(X_train_23, y_train_23)

    #Feature importance for tree-based models (Random Forest, Decision Trees,␣
 ↪Gradient Boosting)
    if hasattr(model, 'feature_importances_'):
        importance = model.feature_importances_

    #Feature importance for linear models (Logistic Regression)
```

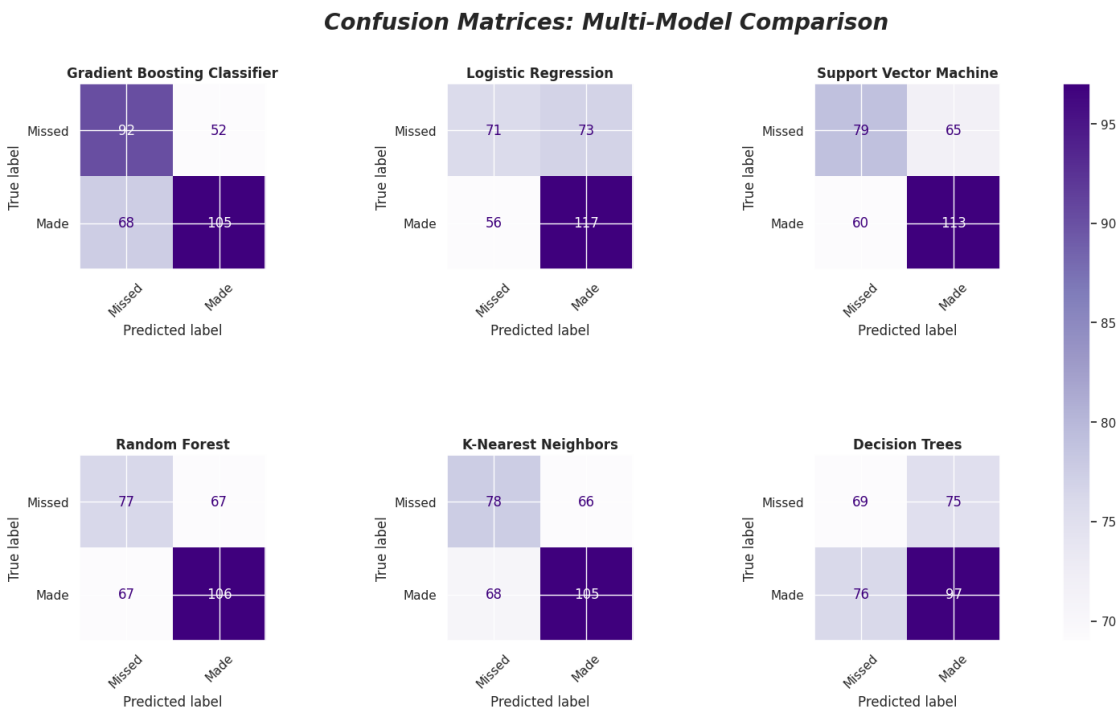
```
    elif hasattr(model, 'coef_'):
        importance = np.abs(model.coef_[0])

    #Permutation importance for models without feature importance (KNN and SVM)
    else:
        result = permutation_importance(model, X_train_23, y_train_23,␣
↪n_repeats=10, random_state=42)
        importance = result.importances_mean

    #Normalize the importance values for better comparison
    if importance is not None:

        importance = importance / np.sum(importance)

        #Plot the bar chart
        sns.barplot(x=importance, y=X_train_df_23.columns, ax=axes[i],␣
↪orient='h',
                    width = 0.75, edgecolor = 'white', linewidth=4)

        #Customize titles
        axes[i].set_title(f'{name}', fontsize=15, fontweight='bold')

        #Customize x-axis
        axes[2].set_xlabel('\nNormalized Importance', fontsize=15,␣
↪fontweight='bold', ha='left')
        axes[0].tick_params(left=True)

        #Customize y-axis
        axes[i].set_yticks(np.arange(len(importance)))
        axes[0].set_yticklabels(('LOC_X', 'LOC_Y', 'SHOT_DISTANCE'),␣
↪rotation=55, fontsize=11)
        axes[0].set_ylabel('Features\n', fontsize=15, fontweight='bold')
        axes[i].tick_params(bottom=True)

#Show plot
fig.suptitle("Feature & Permutation Importance\n", fontsize=23,␣
↪fontweight='bold', fontstyle='oblique')
plt.show()
```



Comparison Matrces w/Default Parameters

[30]:
```
################################ Confusion Matrix 2023␣
↪#######################################
#Creating subplots
fig, axes = plt.subplots(2, 3, figsize=(20, 9))
class_models = {"Gradient Boosting Classifier": GradientBoostingClassifier(),
                "Logistic Regression": LogisticRegression(),
                "Support Vector Machine": SVC(),
                "Random Forest": RandomForestClassifier(),
                "K-Nearest Neighbors": KNeighborsClassifier(),
                "Decision Trees": DecisionTreeClassifier()}


#Creating matrices
for i, (key, model) in enumerate(class_models.items()):
    y_pred_23 = model.fit(X_train_23, y_train_23).predict(X_test_23)
    disp = ConfusionMatrixDisplay(confusion_matrix=confusion_matrix(y_test_23,␣
↪y_pred_23),
                                  display_labels=['Missed', 'Made'])
    disp.plot(ax=axes[i // 3, i % 3], xticks_rotation=45, cmap='Purples')
    disp.ax_.set_title(key, fontsize=12, fontweight='bold')
    disp.im_.colorbar.remove()
    disp.ax_.set_xlabel('Predicted label')
    if i // 3 != 1:
        disp.ax_.set_ylabel('True label')

#Customizing plot
plt.subplots_adjust(wspace=0.40, hspace=1)
fig.suptitle("Confusion Matrices: Multi-Model Comparison", fontsize=20,
             fontweight='bold', fontstyle='oblique', ha='center')
```

```
fig.colorbar(disp.im_, ax=axes)
fig.set_tight_layout(False)
fig.show()
```



Applying k-Fold Cross Validations

[31]:
```
############################## Applying k-Fold Cross Validation 2023␣
↪####################

#Creating a dictionary of the models we'll be using
class_models = {"Logisitic Regression": LogisticRegression(),
                "Random Forest Classifier": RandomForestClassifier(),
                "K-nearest neighbors": KNeighborsClassifier(),
                "Decision Tree Classifier": DecisionTreeClassifier(),
                "Gradient Boosting Classifier": GradientBoostingClassifier(),
                "Support Vector Machine": SVC()}

# Train and evaluate each classifier
results = {'Model': [], 'Average Accuracy': [], 'Standard Deviation': []}

for clf_name, clf in class_models.items():
    # Train the classifier
    clf.fit(X_train_23, y_train_23)
```

```
    # Evaluate the classifier
    accuracies = cross_val_score(estimator = clf, X = X_train_23, y =␣
 ↪y_train_23, cv = 10)
    accuracy = round((accuracies.mean()*100), 4)
    sd = round((accuracies.std()*100), 4)

    # Save results
    results['Model'].append(clf_name)
    results['Average Accuracy'].append(accuracy)
    results['Standard Deviation'].append(sd)

# Create a DataFrame to display the results
results_df = pd.DataFrame(results)
results_df.set_index('Model', inplace=True)
results_df.sort_values(by='Average Accuracy', ascending=False).style.
 ↪background_gradient(cmap='Greens')
```

[31]: <pandas.io.formats.style.Styler at 0x7fb54455ca40>

Hyperparameter Tuning for our Best Performing Model

[32]:
```
############################## Using GridSearchCV ####################
#Define the parameter grid for GridSearchCV
param_grid = {'n_estimators': [50, 100, 200],
              'learning_rate': [0.01, 0.1, 0.2, 0.3, 0.4, 0.5],
              'max_depth': [1, 3, 5, 7]}

#Run the Gradient Boosting model
gb_model = GradientBoostingClassifier()

#Initialize GridSearchCV
grid_search = GridSearchCV(estimator=gb_model, param_grid=param_grid, cv=10,␣
 ↪scoring='accuracy', n_jobs=-1)

#Fit the model to the training data using GridSearchCV
grid_search.fit(X_train_23, y_train_23)

#Get the best parameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

#Make predictions on the test set using the best model
y_pred_best_23 = best_model.predict(X_test_23)

#Evaluate the best model
accuracy_best = accuracy_score(y_test_23, y_pred_best_23)
```

```
#Print results
print("Best Parameters (Grid Search):", best_params)
print(f"Best Model Accuracy (Grid Search): {accuracy_best}")
```

```
Best Parameters (Grid Search): {'learning_rate': 0.01, 'max_depth': 1,
'n_estimators': 50}
Best Model Accuracy (Grid Search): 0.6182965299684543
```

Gradient Boosted Trees is the model we'll use for both data sets as it boasts the highest Accuracy and Precision score after tuning all the classification models tested. It's important as well to point out feature importance with how Gradient Boosting and Random Forest both hold shot coordinate features 'LOC_X' and 'LOC_Y' at a much higher relevance than Logistic Regression which holds 'SHOT_DISTANCE' as the most important features in its prediction model.

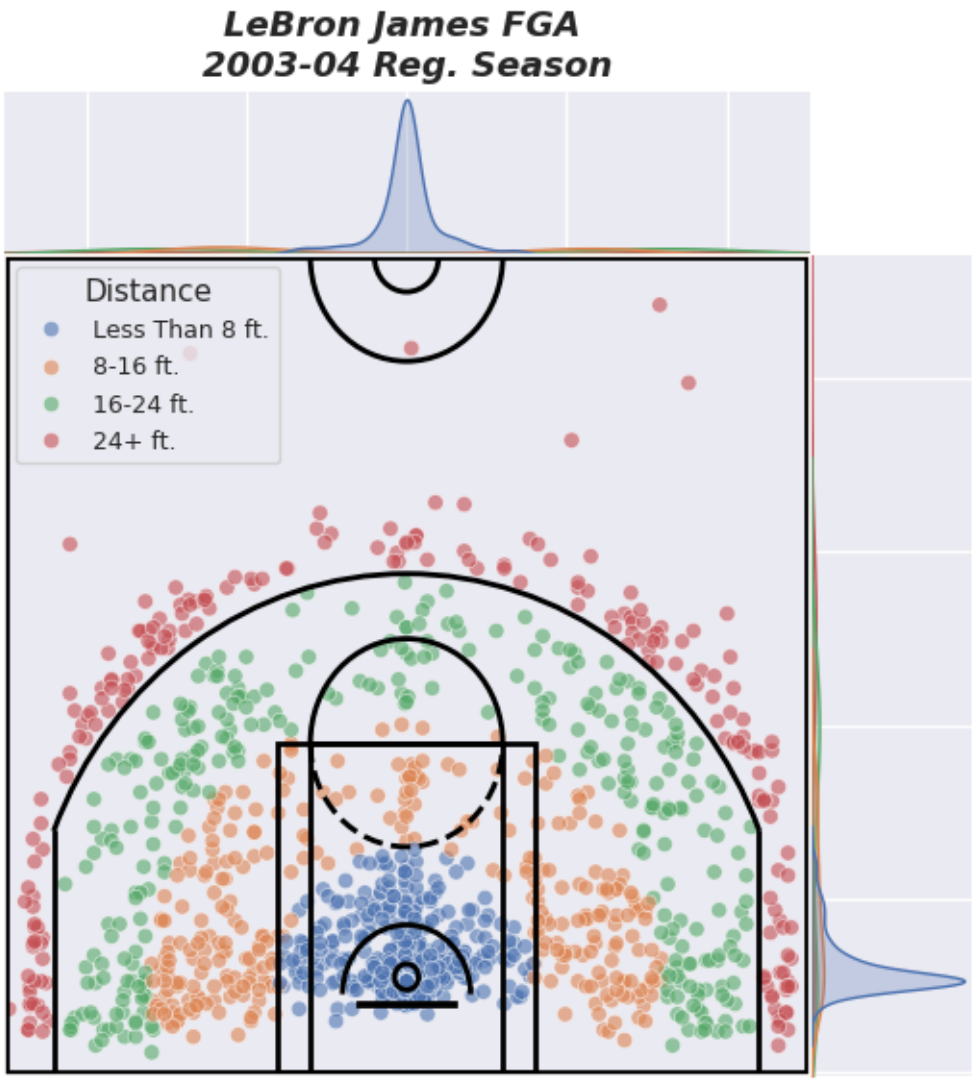
13 4.3 LeBron James 2004

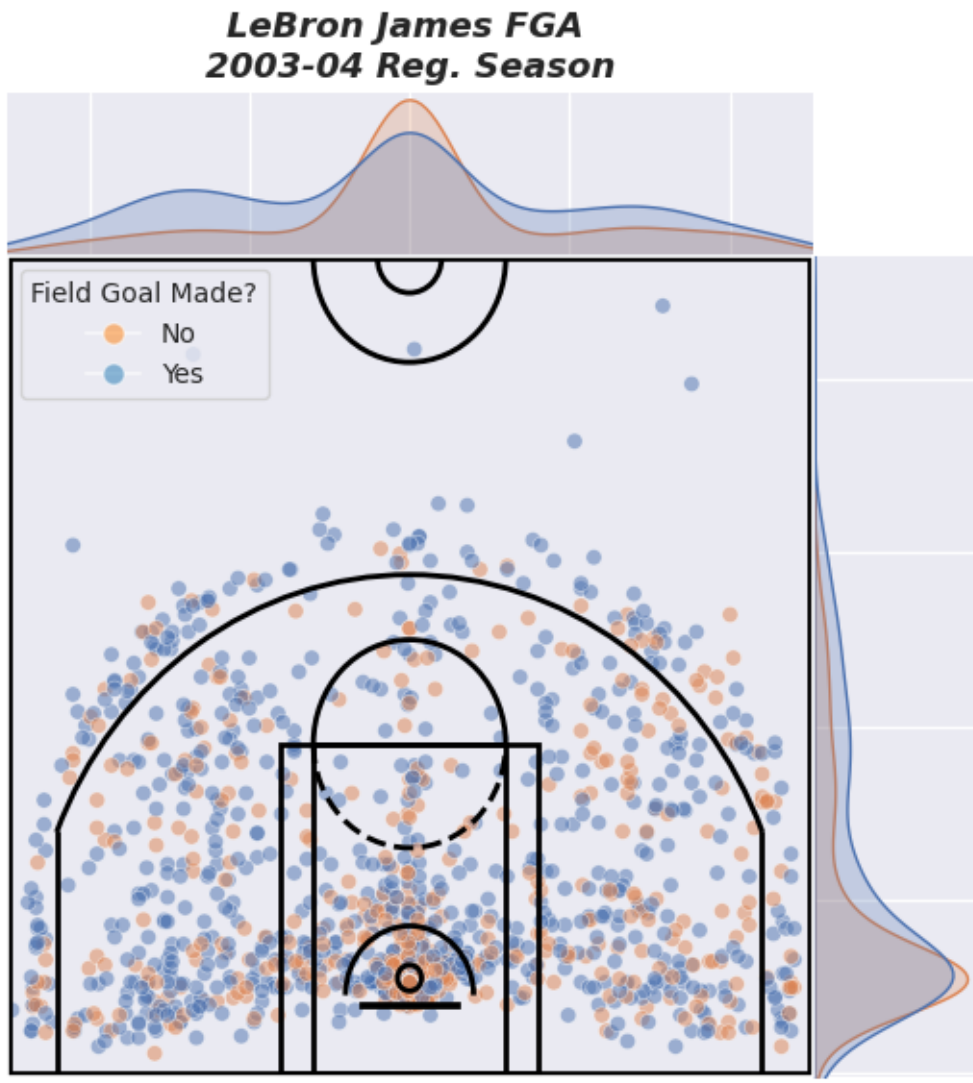
[33]:
```
#Setting up seaborn plot
plt.figure(figsize=(15,9.4))
sns.set_theme(style="darkgrid")
sns.set_color_codes()


#################################### DISTANCE␣
↪########################################
#Plotting
ax = sns.jointplot(data = lebron_04,
                   x='LOC_X',
                   y='LOC_Y',
                   kind='scatter',
                   hue='ZONE_RANGE',
                   hue_order=['Less Than 8 ft.', '8-16 ft.', '16-24 ft.', '24+␣
↪ft.'],
                   space=0,
                   alpha=0.6)


#Court and axes
draw_court(outer_lines=True, color="black")
cur_axes = plt.gca()
cur_axes.axes.get_xaxis().set_visible(False)
cur_axes.axes.get_yaxis().set_visible(False)
plt.xlim(-251, 252)
plt.ylim(-2, 471)

#Title
plt.title('LeBron James FGA \n2003-04 Reg. Season',
          fontsize=13,
```

```
          loc='center',
          fontweight='bold',
          y=1.2,
          fontstyle='oblique')

#Legend
plt.legend(title='Distance',
          loc='upper left',
          title_fontsize= 11,
          fontsize = 9)

################################### FIELD GOAL MADE?␣
↪#####################################
#Plotting
g = sns.jointplot(data = lebron_04,
                  x='LOC_X',
                  y='LOC_Y',
                  kind='scatter',
                  hue='SHOT_MADE',
                  space=0,
                  alpha=0.5)

#Court and axes
draw_court(outer_lines=True, color="black")
cur_axes = plt.gca()
cur_axes.axes.get_xaxis().set_visible(False)
cur_axes.axes.get_yaxis().set_visible(False)
plt.xlim(-251, 252)
plt.ylim(-2, 471)

#Title
plt.title('LeBron James FGA \n2003-04 Reg. Season',
          fontsize=13,
          loc='center',
          fontweight='bold',
          y=1.2,
          fontstyle='oblique')

from matplotlib.lines import Line2D
#Legend
legend_labels = [Line2D([0], [0], marker='o', color='w', markerfacecolor='tab:
↪orange', markersize=8, alpha=0.5, label='No'),
                 Line2D([0], [0], marker='o', color='w', markerfacecolor='tab:
↪blue', markersize=8, alpha=0.5,  label='Yes')]
g.ax_joint.legend(handles=legend_labels, title='Field Goal Made?', loc='upper␣
↪left', title_fontsize=10, fontsize=10)
```

```
plt.show()
```

```
<Figure size 1500x940 with 0 Axes>
```

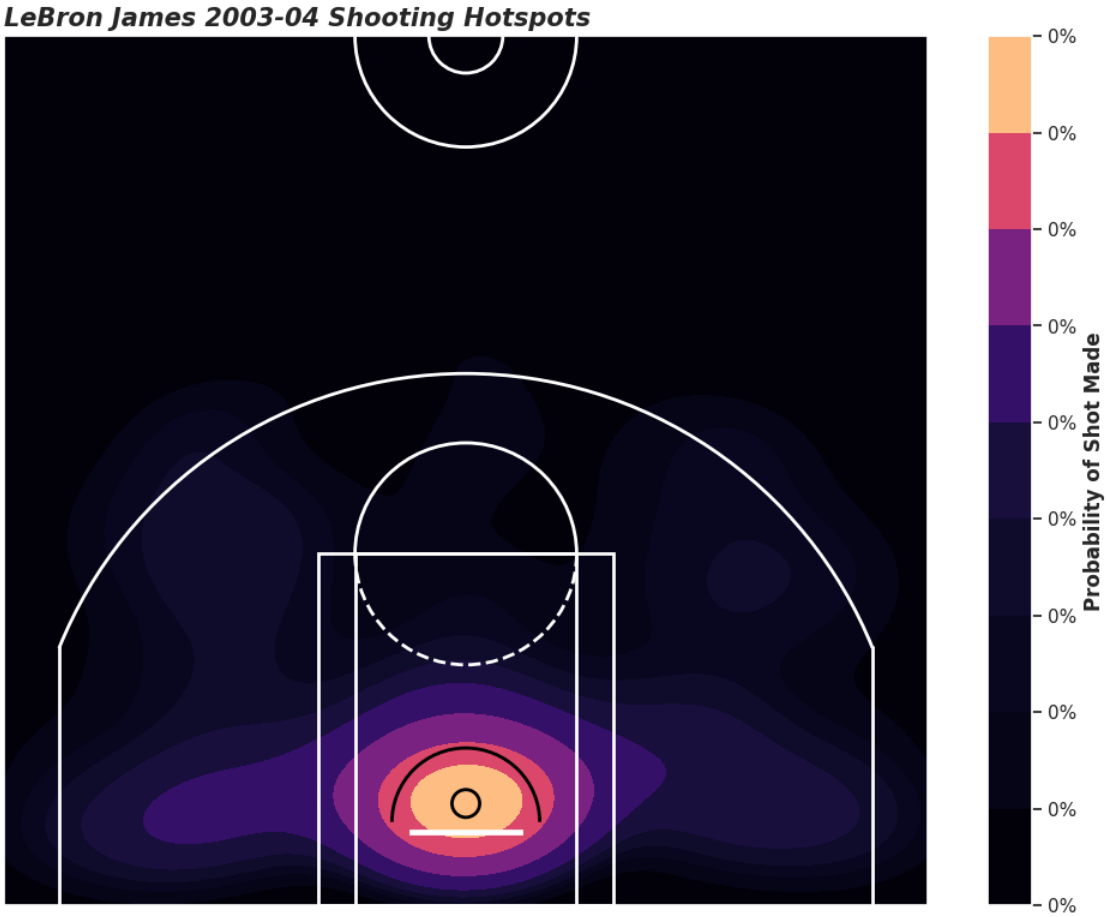




Now using our Gradient Boosted model let's look at a kdeplot for location mapping to get a better sense of how Lebron's shooting habits and hotspots

[34]:
```
#Player data
X = lebron04_ml.iloc[:, :-1].values
y = lebron04_ml.iloc[:, -1].values

#Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,␣
↪random_state=42)

#Training Gradient Boosting Classifier
gb = GradientBoostingClassifier(n_estimators=200, learning_rate=0.01,␣
↪max_depth=3)
```

```
gb.fit(X_train, y_train)

#Probabilities for each shot
lebron04_ml['shot_probability'] = gb.predict_proba(lebron04_ml.iloc[:, :-1].
 ↪values)[:, 1]

#Basketball court and shooting hotspots using KDE contour plot
def plot_shooting_hotspots(data):
    #Plotting basketball court
    plt.figure(figsize=(13, 9.4))
    sns.set_theme(style="darkgrid")

    #KDE contour plot
    ax = sns.kdeplot(x=data["LOC_X"], y=data["LOC_Y"], fill=True, cmap="magma",␣
 ↪levels=10, thresh=0, common_norm=False)

    #Colorbar with percentage labels
    cbar = plt.colorbar(ax.collections[0], label='Shooting Probability',␣
 ↪format=FuncFormatter(lambda x, _: f'{int(x*100)}%'))

    plt.title('LeBron James 2003-04 Shooting Hotspots', fontsize=15,␣
 ↪fontweight='bold', fontstyle='oblique', loc='left')
    plt.xticks([])
    plt.yticks([])
    draw_court(outer_lines=False, color="white", lw=2)
    cur_axes = plt.gca()
    cur_axes.axes.get_xaxis().set_visible(False)
    cur_axes.axes.get_yaxis().set_visible(False)
    plt.xlim(-250, 250)
    plt.ylim(0, 470)
    plt.gca().set_aspect('equal', adjustable='box')  # Keep the court aspect␣
 ↪ratio

    #Adjust the colorbar position if needed
    cbar.set_label('Probability of Shot Made', fontweight='bold')
    plt.show()

#Shooting hotspots on the court
plot_shooting_hotspots(lebron04_ml)
```

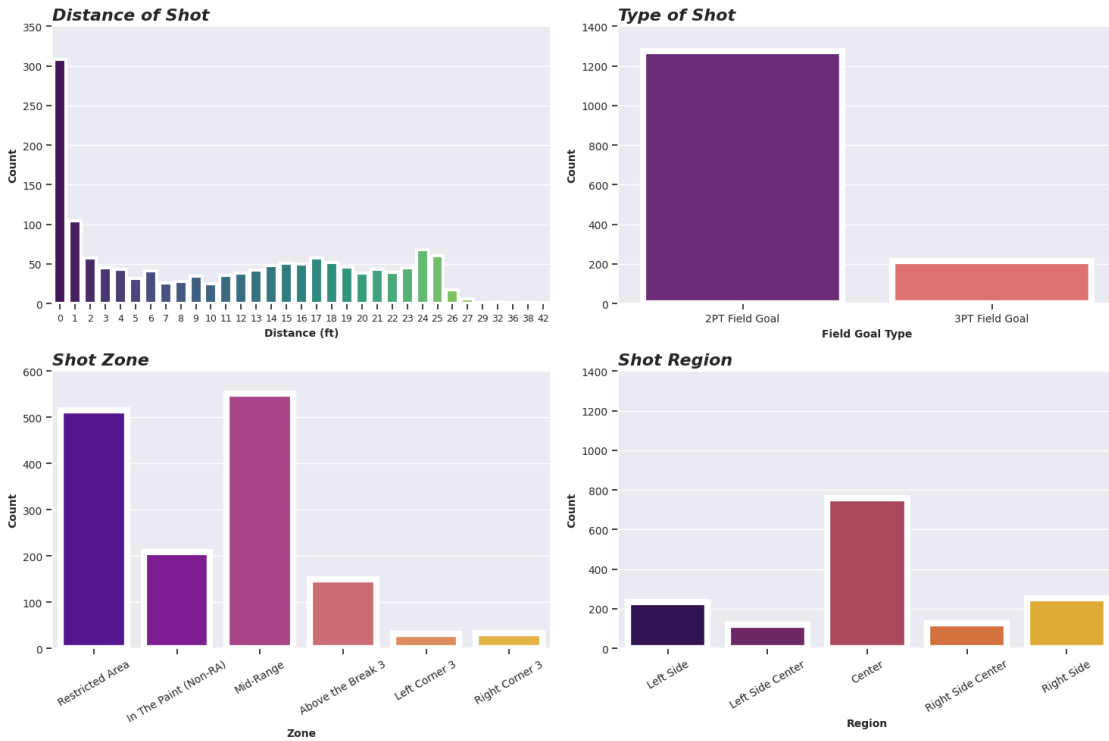


[35]:
```
#Setting up subplots
fig, ax = plt.subplots(nrows=2, ncols=2)
sns.set_theme(style="darkgrid")

#Defining our Dataframe
df=lebron_04

#Plot 1
sns.countplot(x="SHOT_DISTANCE", data=df, ax=ax[0, 0], palette='viridis',␣
 ↪edgecolor = "white", linewidth=3)
ax[0, 0].set_title('Distance of Shot', fontweight='bold', fontstyle='oblique',␣
 ↪loc="left", size=16)
ax[0, 0].set_xlabel('Distance (ft)', size=10, fontweight='bold')
ax[0, 0].set_ylabel('Count', size=10, fontweight='bold')
ax[0, 0].set_xticks(ax[0, 0].get_xticks())
ax[0, 0].set_yticks(ax[0, 0].get_yticks())
ax[0, 0].set_xticklabels(ax[0, 0].get_xticklabels(), rotation=0, ha='center',␣
 ↪size=9)
```

```
ax[0, 0].set_yticklabels(ax[0, 0].get_yticklabels(), rotation=0, ha='right',␣
↪size=10)
ax[0, 0].tick_params(bottom=True, left=True)

#Plot 2
sns.countplot(x="SHOT_TYPE", data=df, ax=ax[0, 1], palette='magma', edgecolor =␣
↪"white", linewidth=6,
              order=['2PT Field Goal', '3PT Field Goal'])
ax[0, 1].set_title('Type of Shot', fontweight='bold', fontstyle='oblique',␣
↪loc="left", size=16)
ax[0, 1].set_xlabel('Field Goal Type', size=10, fontweight='bold')
ax[0, 1].set_ylabel('Count', size=10, fontweight='bold')
ax[0, 1].set_xticks(ax[0, 1].get_xticks())
ax[0, 1].set_yticks(ax[0, 1].get_yticks())
ax[0, 1].set_xticklabels(ax[0, 1].get_xticklabels(), rotation=0, ha='center',␣
↪size=10)
ax[0, 1].set_yticklabels(ax[0, 1].get_yticklabels(), rotation=0, ha='right',␣
↪size=10)
ax[0, 1].tick_params(bottom=True, left=True)

#Plot 3
sns.countplot(x="BASIC_ZONE", data=df, ax=ax[1, 0], palette='plasma',edgecolor␣
↪= "white", linewidth=6,
              order=['Restricted Area', 'In The Paint (Non-RA)',␣
↪'Mid-Range','Above the Break 3', 'Left Corner 3', 'Right Corner 3'])
ax[1, 0].set_title('Shot Zone', fontweight='bold', fontstyle='oblique',␣
↪loc="left", size=16)
ax[1, 0].set_xlabel('Zone', size=10, fontweight='bold')
ax[1, 0].set_ylabel('Count', size=10, fontweight='bold')
ax[1, 0].set_xticks(ax[1, 0].get_xticks())
ax[1, 0].set_yticks(ax[1, 0].get_yticks())
ax[1, 0].set_xticklabels(ax[1, 0].get_xticklabels(), rotation=30, ha='center',␣
↪size=10)
ax[1, 0].set_yticklabels(ax[1, 0].get_yticklabels(), rotation=0, ha='right',␣
↪size=10)
ax[1, 0].tick_params(bottom=True, left=True)

#Plot 4
sns.countplot(x="ZONE_NAME", data=df, ax=ax[1, 1], palette='inferno', edgecolor␣
↪= "white", linewidth=6,
              order=['Left Side', 'Left Side Center', 'Center', 'Right Side␣
↪Center', 'Right Side'])
ax[1, 1].set_title('Shot Region', fontweight='bold', fontstyle='oblique',␣
↪loc="left", size=16)
ax[1, 1].set_xlabel('Region', size=10, fontweight='bold')
ax[1, 1].set_ylabel('Count', size=10, fontweight='bold')
```

```
ax[1, 1].set_xticks(ax[1, 1].get_xticks())
ax[1, 1].set_yticks(ax[0, 1].get_yticks())
ax[1, 1].set_xticklabels(ax[1, 1].get_xticklabels(), rotation=30, ha='center',␣
 ↪size=10)
ax[1, 1].set_yticklabels(ax[1, 1].get_yticklabels(), rotation=0, ha='right',␣
 ↪size=10)
ax[1, 1].tick_params(bottom=True, left=True)

# Adjust layout
plt.tight_layout()

# Show the plots
plt.show()
```

```
/tmp/ipython-input-3695176652.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="SHOT_DISTANCE", data=df, ax=ax[0, 0], palette='viridis',
edgecolor = "white", linewidth=3)
/tmp/ipython-input-3695176652.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="SHOT_TYPE", data=df, ax=ax[0, 1], palette='magma', edgecolor
= "white", linewidth=6,
/tmp/ipython-input-3695176652.py:32: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="BASIC_ZONE", data=df, ax=ax[1, 0], palette='plasma',edgecolor
= "white", linewidth=6,
/tmp/ipython-input-3695176652.py:44: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="ZONE_NAME", data=df, ax=ax[1, 1], palette='inferno',
edgecolor = "white", linewidth=6,
```



Coaches during and after the 2003-04 season would evaluate LeBron's shot diet as that of a strong finisher at the rim with a deadly baseline jump shot from both left and right. These findings indicate optimal defensive locations to guard LeBron due to his success in those regions.

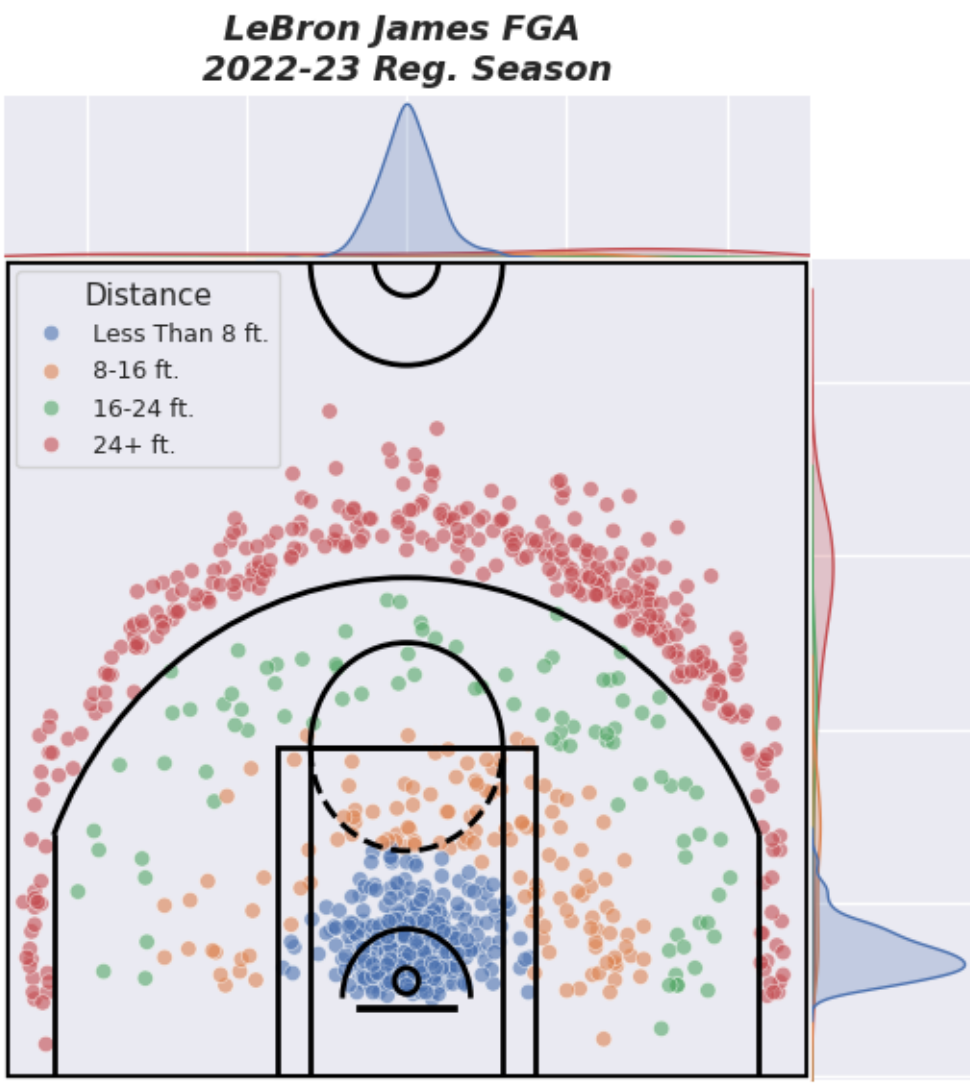
14 4.4 LeBron James 2023

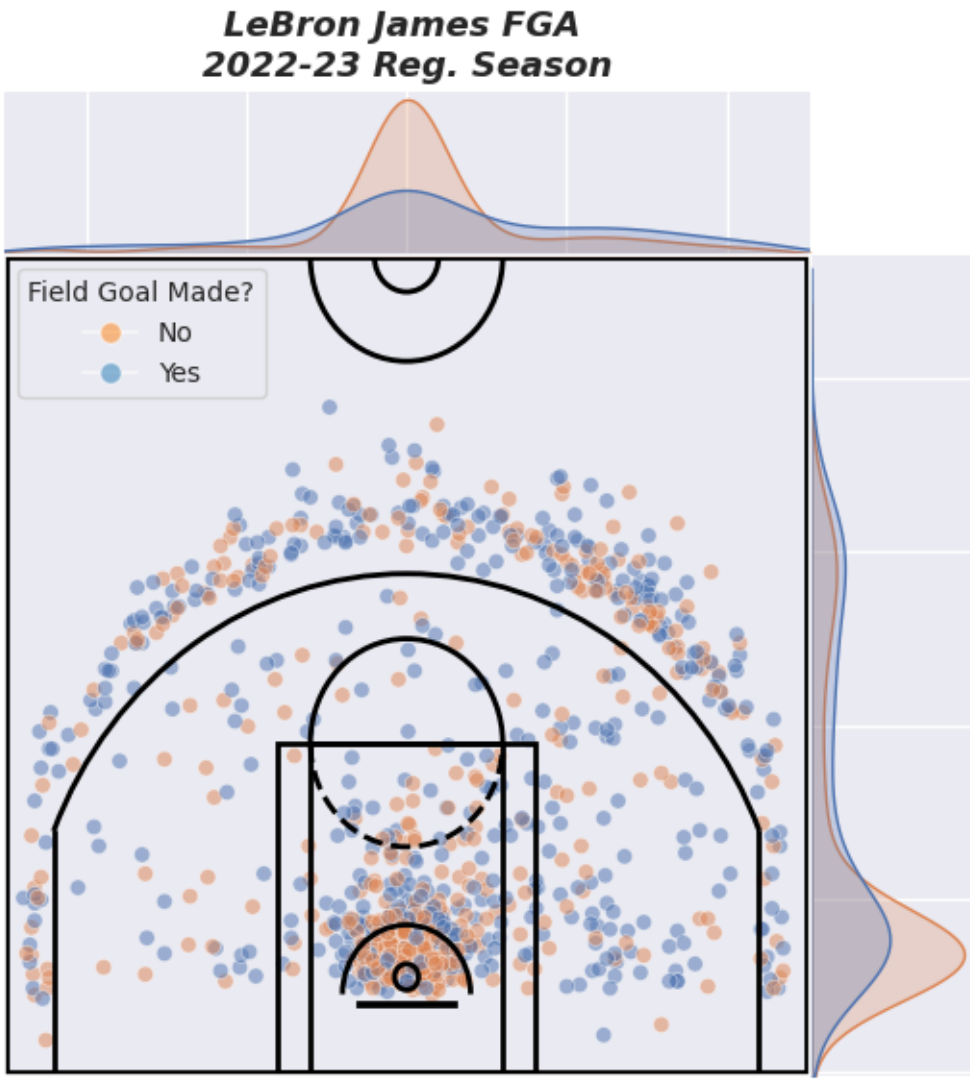
[36]:
```
plt.figure(figsize=(15,9.4))
sns.set_theme(style="darkgrid")
sns.set_color_codes()

#################################### DISTANCE␣
↪########################################
#Plotting
sns.jointplot(data = lebron_23,
              x='LOC_X',
              y='LOC_Y',
              kind='scatter',
              hue='ZONE_RANGE',
              hue_order=['Less Than 8 ft.', '8-16 ft.', '16-24 ft.', '24+ ft.'],
              space=0,
              alpha=0.6)

#Court and axis
```

```
draw_court(outer_lines=True, color="black")
cur_axes = plt.gca()
cur_axes.axes.get_xaxis().set_visible(False)
cur_axes.axes.get_yaxis().set_visible(False)
plt.xlim(-251, 252)
plt.ylim(-2, 471)

#Title
plt.title('LeBron James FGA \n2022-23 Reg. Season',
          fontsize=13,
          loc='center',
          fontweight='bold',
          y=1.2,
          fontstyle='oblique')

#Legend
plt.legend(title='Distance',
          loc='upper left',
          title_fontsize= 11,
          fontsize = 9)

################################## FIELD GOAL MADE?␣
↪#####################################
#Plotting
g = sns.jointplot(data = lebron_23,
                  x='LOC_X',
                  y='LOC_Y',
                  kind='scatter',
                  hue='SHOT_MADE',
                  space=0,
                  alpha=0.5,
                  legend=False)

#Court and axes
draw_court(outer_lines=True, color="black")
cur_axes = plt.gca()
cur_axes.axes.get_xaxis().set_visible(False)
cur_axes.axes.get_yaxis().set_visible(False)
plt.xlim(-251, 252)
plt.ylim(-2, 471)

#Title
plt.title('LeBron James FGA \n2022-23 Reg. Season',
          fontsize=13,
          loc='center',
          fontweight='bold',
          y=1.2,
```

```
          fontstyle='oblique')

#Legend
legend_labels = [Line2D([0], [0], marker='o', color='w', markerfacecolor='tab:
 ↪orange', markersize=8, alpha=0.5, label='No'),
                 Line2D([0], [0], marker='o', color='w', markerfacecolor='tab:
 ↪blue', markersize=8, alpha=0.5,  label='Yes')]
g.ax_joint.legend(handles=legend_labels, title='Field Goal Made?', loc='upper␣
 ↪left', title_fontsize=10, fontsize=10)


plt.show()
```

```
<Figure size 1500x940 with 0 Axes>
```

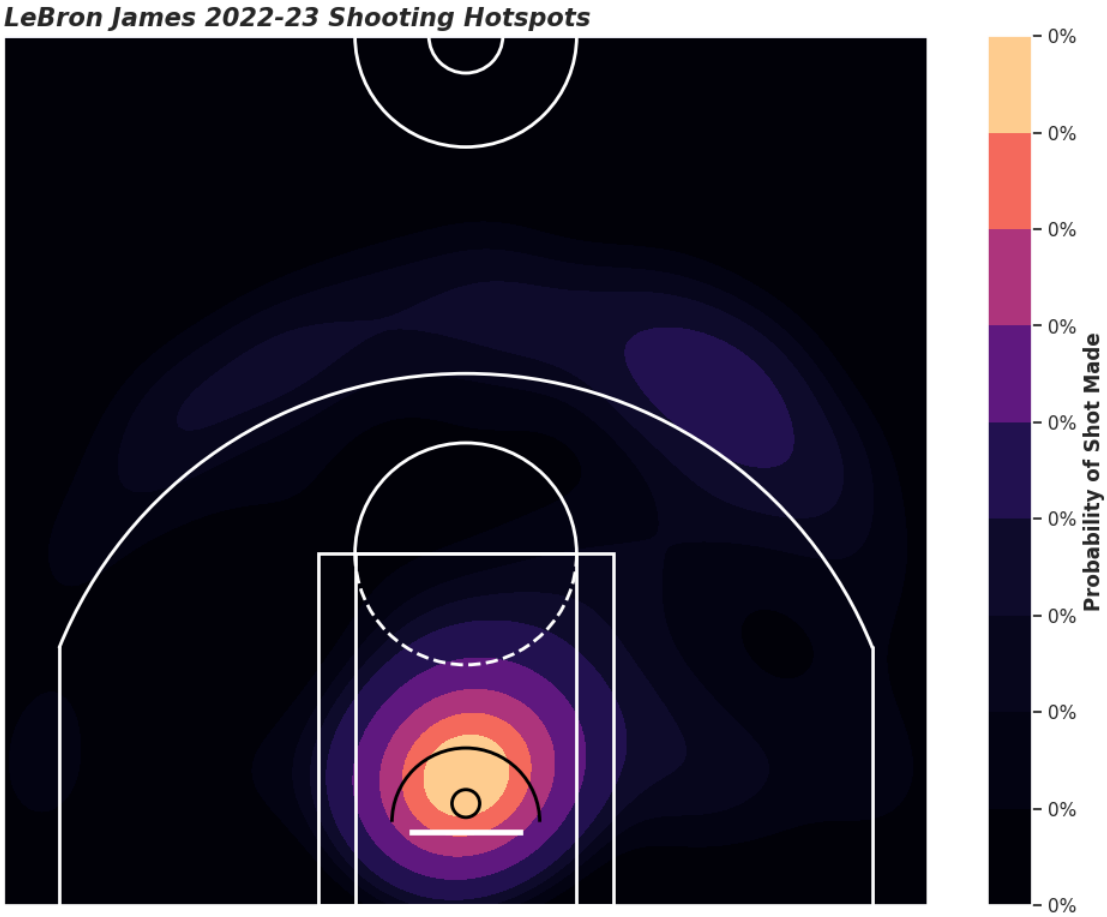




We see a clear difference in Lebron's shooting habits with the distribution graph on the right showing us a higher volume of 3-Pointers (24+ft) shot by Lebron in the *2022-23* season in comparison to his rookie season in *2003-04*.

[37]:
```
#Simulated player data
X = lebron23_ml.iloc[:, :-1].values
y = lebron23_ml.iloc[:, -1].values

#Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,␣
↪random_state=15)
```

```python
#Train a Gradient Boosting Classifier
gb = GradientBoostingClassifier(n_estimators=200, learning_rate=0.01,␣
↪max_depth=3)
gb.fit(X_train, y_train)

#Predict probabilities for each shot
lebron23_ml['shot_probability'] = gb.predict_proba(lebron23_ml.iloc[:, :-1].
↪values)[:, 1]

#Function to plot the basketball court and shooting hotspots using KDE contour␣
↪plot
def plot_shooting_hotspots(data):
    # Plot the basketball court
    plt.figure(figsize=(13, 9.4))

    #Create KDE contour plot
    ax = sns.kdeplot(x=data["LOC_X"], y=data["LOC_Y"], fill=True, cmap="magma",␣
↪levels=10, thresh=0, common_norm=False)

    #Explicitly show the colorbar with percentage labels
    cbar = plt.colorbar(ax.collections[0], label='Shooting Probability',␣
↪format=FuncFormatter(lambda x, _: f'{int(x*100)}%'))

    plt.title('LeBron James 2022-23 Shooting Hotspots', fontsize=15,␣
↪fontweight='bold', fontstyle='oblique', loc='left')
    plt.xticks([])
    plt.yticks([])
    draw_court(outer_lines=False, color="white", lw=2)
    cur_axes = plt.gca()
    cur_axes.axes.get_xaxis().set_visible(False)
    cur_axes.axes.get_yaxis().set_visible(False)
    plt.xlim(-250, 250)
    plt.ylim(0, 470)
    plt.gca().set_aspect('equal', adjustable='box')

    #Colorbar
    cbar.set_label('Probability of Shot Made', fontweight='bold')
    plt.show()

#Plot the shooting hotspots on the court using KDE contour plot
plot_shooting_hotspots(lebron23_ml)
```

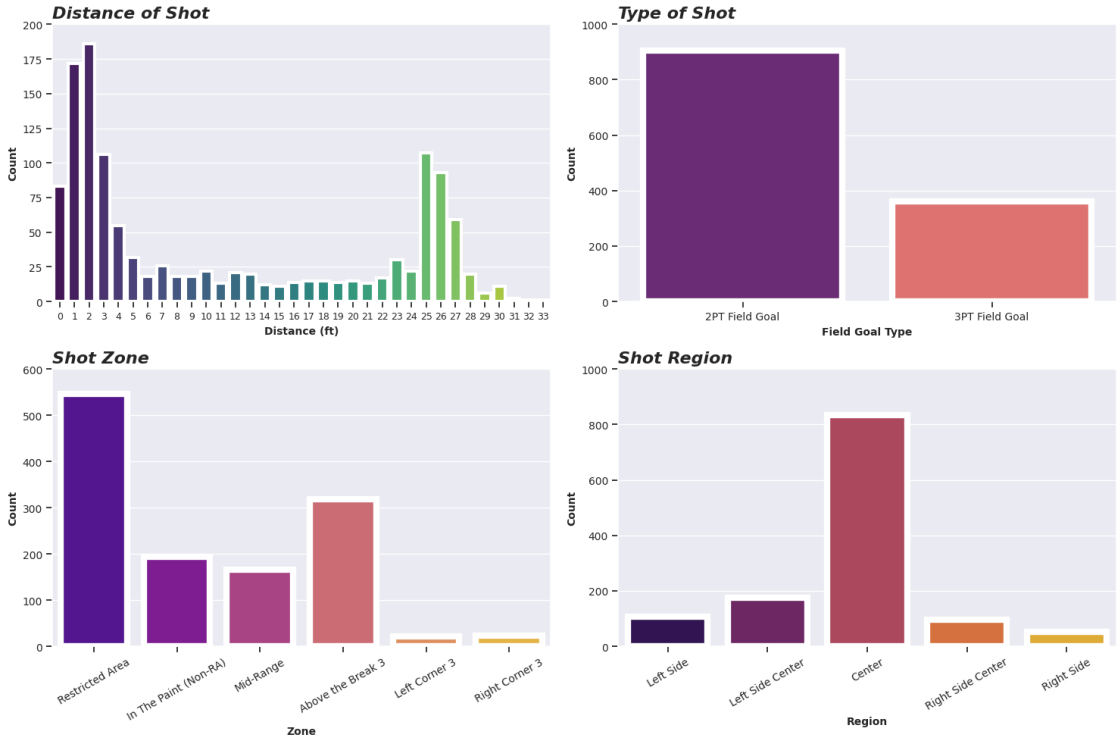


[38]:
```
#Setting up subplots
fig, ax = plt.subplots(nrows=2, ncols=2)
sns.set_theme(style="darkgrid")

#Defining our Dataframe
df=lebron_23

#Plot 1
sns.countplot(x="SHOT_DISTANCE", data=df, ax=ax[0, 0], palette='viridis',␣
↪edgecolor = "white", linewidth=3)
ax[0, 0].set_title('Distance of Shot', fontweight='bold', fontstyle='oblique',␣
↪loc="left", size=16)
ax[0, 0].set_xlabel('Distance (ft)', size=10, fontweight='bold')
ax[0, 0].set_ylabel('Count', size=10, fontweight='bold')
ax[0, 0].set_xticks(ax[0, 0].get_xticks())
ax[0, 0].set_yticks(ax[0, 0].get_yticks())
ax[0, 0].set_xticklabels(ax[0, 0].get_xticklabels(), rotation=0, ha='center',␣
↪size=9)
```

```
ax[0, 0].set_yticklabels(ax[0, 0].get_yticklabels(), rotation=0, ha='right',␣
↪size=10)
ax[0, 0].tick_params(bottom=True, left=True)

#Plot 2
sns.countplot(x="SHOT_TYPE", data=df, ax=ax[0, 1], palette='magma', edgecolor =␣
↪"white", linewidth=6,
              order=['2PT Field Goal', '3PT Field Goal'])
ax[0, 1].set_title('Type of Shot', fontweight='bold', fontstyle='oblique',␣
↪loc="left", size=16)
ax[0, 1].set_xlabel('Field Goal Type', size=10, fontweight='bold')
ax[0, 1].set_ylabel('Count', size=10, fontweight='bold')
ax[0, 1].set_xticks(ax[0, 1].get_xticks())
ax[0, 1].set_yticks(ax[0, 1].get_yticks())
ax[0, 1].set_xticklabels(ax[0, 1].get_xticklabels(), rotation=0, ha='center',␣
↪size=10)
ax[0, 1].set_yticklabels(ax[0, 1].get_yticklabels(), rotation=0, ha='right',␣
↪size=10)
ax[0, 1].tick_params(bottom=True, left=True)

#Plot 3
sns.countplot(x="BASIC_ZONE", data=df, ax=ax[1, 0], palette='plasma',edgecolor␣
↪= "white", linewidth=6,
              order=['Restricted Area', 'In The Paint (Non-RA)',␣
↪'Mid-Range','Above the Break 3', 'Left Corner 3', 'Right Corner 3'])
ax[1, 0].set_title('Shot Zone', fontweight='bold', fontstyle='oblique',␣
↪loc="left", size=16)
ax[1, 0].set_xlabel('Zone', size=10, fontweight='bold')
ax[1, 0].set_ylabel('Count', size=10, fontweight='bold')
ax[1, 0].set_xticks(ax[1, 0].get_xticks())
ax[1, 0].set_yticks(ax[1, 0].get_yticks())
ax[1, 0].set_xticklabels(ax[1, 0].get_xticklabels(), rotation=30, ha='center',␣
↪size=10)
ax[1, 0].set_yticklabels(ax[1, 0].get_yticklabels(), rotation=0, ha='right',␣
↪size=10)
ax[1, 0].tick_params(bottom=True, left=True)

#Plot 4
sns.countplot(x="ZONE_NAME", data=df, ax=ax[1, 1], palette='inferno', edgecolor␣
↪= "white", linewidth=6,
              order=['Left Side', 'Left Side Center', 'Center', 'Right Side␣
↪Center', 'Right Side'])
ax[1, 1].set_title('Shot Region', fontweight='bold', fontstyle='oblique',␣
↪loc="left", size=16)
ax[1, 1].set_xlabel('Region', size=10, fontweight='bold')
ax[1, 1].set_ylabel('Count', size=10, fontweight='bold')
```

```
ax[1, 1].set_xticks(ax[1, 1].get_xticks())
ax[1, 1].set_yticks(ax[0, 1].get_yticks())
ax[1, 1].set_xticklabels(ax[1, 1].get_xticklabels(), rotation=30, ha='center',␣
 ↪size=10)
ax[1, 1].set_yticklabels(ax[1, 1].get_yticklabels(), rotation=0, ha='right',␣
 ↪size=10)
ax[1, 1].tick_params(bottom=True, left=True)

# Adjust layout
plt.tight_layout()

# Show the plots
plt.show()
```

```
/tmp/ipython-input-4108759749.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="SHOT_DISTANCE", data=df, ax=ax[0, 0], palette='viridis',
edgecolor = "white", linewidth=3)
/tmp/ipython-input-4108759749.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="SHOT_TYPE", data=df, ax=ax[0, 1], palette='magma', edgecolor
= "white", linewidth=6,
/tmp/ipython-input-4108759749.py:32: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="BASIC_ZONE", data=df, ax=ax[1, 0], palette='plasma',edgecolor
= "white", linewidth=6,
/tmp/ipython-input-4108759749.py:44: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="ZONE_NAME", data=df, ax=ax[1, 1], palette='inferno',
edgecolor = "white", linewidth=6,
```



LeBron's evolution is very interesting to see as he still attacks the basket in his 21st season with the same tenacity as when he did in his rookie season with the biggest change coming in his willingness to shoot more 3s, especially from that top right position which is his favorite.

15 5. *Conclusion*

The NBA today finds itself in the middle of one of its most defining eras with the 3-point boom. With players like Stephen Curry being the poster child for this new era, the staggering effects of such a paradigm shift in the league but also in basketball as a whole have had indescribable impacts on how basketball at all levels is played, coached, and even examined. With the upward trend of NBA players attempting and making more 3-point shots from the beginning of the 21st century to the present being clear, the offensive capabilities of NBA teams (made evident with higher than-ever final point totals and player points per game) have reached levels never before seen. A change like this however doesn't come with the impact of just one player like Stephen Curry but with there being external factors or influences on and off the court which radically changed systems within the league.

Starting around the 2013-14 and 2014-2015 seasons behind the scenes the NBA was also going through another revolution in the form of the analytics boom. Teams like the Houston Rockets with offensive-minded Head Coach Mike D'Antoni and star player James Harden were the faces of this new offensive-minded movement. If one individual and their team could be labeled as leading the way behind the scenes it was Houston Rockets General Manager Daryl Morey. With his analytics-focused basketball philosophy, labeled "Moreyball," Morey pushed for his team to concentrate on more "statistical sound" shot attempts which focused less on mid-range shots and

more on 3s and shots closer to the basket.

Morey's philosophy sent shockwaves through the NBA and other teams followed suit. Using data similar to how Morey did, in this study, we showcased how vital it is for coaches, trainers, and higher-up executives within teams to find and follow important trends. Such findings can then be acted upon to make well-informed decisions to prepare for the future of not just the NBA but of all of basketball.

[13]:
```
!apt-get install -y texlive-xetex texlive-fonts-recommended texlive-latex-extra
```

```
Reading package lists… Done
Building dependency tree… Done
Reading state information… Done
texlive-fonts-recommended is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-latex-extra set to manually installed.
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 35 not upgraded.
```

[19]:
```
!pip install nbconvert
```

```
Requirement already satisfied: nbconvert in /usr/local/lib/python3.12/dist-
packages (7.16.6)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (4.13.5)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-
packages (from bleach[css]!=5.0.0->nbconvert) (6.2.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (5.8.1)
Requirement already satisfied: jupyterlab-pygments in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (3.0.2)
Requirement already satisfied: mistune<4,>=2.0.3 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (3.1.4)
Requirement already satisfied: nbclient>=0.5.0 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (0.10.2)
Requirement already satisfied: nbformat>=5.7 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (25.0)
Requirement already satisfied: pandocfilters>=1.4.1 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (1.5.1)
Requirement already satisfied: pygments>=2.4.1 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (2.19.2)
```

```
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (5.7.1)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-
packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert)
(1.4.0)
Requirement already satisfied: platformdirs>=2.5 in
/usr/local/lib/python3.12/dist-packages (from jupyter-core>=4.7->nbconvert)
(4.4.0)
Requirement already satisfied: jupyter-client>=6.1.12 in
/usr/local/lib/python3.12/dist-packages (from nbclient>=0.5.0->nbconvert)
(7.4.9)
Requirement already satisfied: fastjsonschema>=2.15 in
/usr/local/lib/python3.12/dist-packages (from nbformat>=5.7->nbconvert) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in
/usr/local/lib/python3.12/dist-packages (from nbformat>=5.7->nbconvert) (4.25.1)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-
packages (from beautifulsoup4->nbconvert) (2.8)
Requirement already satisfied: typing-extensions>=4.0.0 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4->nbconvert)
(4.15.0)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-
packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/usr/local/lib/python3.12/dist-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (2025.4.1)
Requirement already satisfied: referencing>=0.28.4 in
/usr/local/lib/python3.12/dist-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.36.2)
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/dist-
packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.27.1)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-
packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (0.4)
Requirement already satisfied: nest-asyncio>=1.5.4 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: pyzmq>=23.0 in /usr/local/lib/python3.12/dist-
packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (26.2.1)
Requirement already satisfied: tornado>=6.2 in /usr/local/lib/python3.12/dist-
packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.4.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.8.2->jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
```

[20]:
```
!jupyter nbconvert --to pdf 3-Point_Shooting_Era_Breakdown.ipynb
```

```
[NbConvertApp] WARNING | pattern '3-Point_Shooting_Era_Breakdown.ipynb' matched
no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and
include the error message in the cell output (the default behaviour is to abort
conversion). This flag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with
default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
```

```
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each
cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True
--TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True
--TemplateExporter.exclude_input=True
--TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the
system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful
for the HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR',
'CRITICAL']
    Default: 30
```

```
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
            ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook',
'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
            or a dotted object name that represents the import path for an
            ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                        results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                        results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
```

```
                                    to output to the directory of each notebook.
To recover
                                    previous default behaviour (outputting to the
current
                                    working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
            This defaults to the reveal CDN, but can be any url pointing to a
copy
            of reveal.js.
            For speaker notes to work, this must be a relative path to a local
            copy of reveal.js: e.g., "reveal.js".
            If a relative path is given, it must be a subdirectory of the
            current directory (from which the server is run).
            See the usage documentation
            (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-
html-slideshow)
            for more details.
    Default: ''
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
            Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

Examples
--------

    The simplest way to use nbconvert is

            > jupyter nbconvert mynotebook.ipynb --to html

            Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown',
'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides',
'webpdf'].

            > jupyter nbconvert --to latex mynotebook.ipynb

            Both HTML and LaTeX support multiple output templates. LaTeX
includes
            'base', 'article' and 'report'.  HTML includes 'basic', 'lab' and
            'classic'. You can specify the flavor of the format used.

            > jupyter nbconvert --to html --template lab mynotebook.ipynb
```

```
            You can also pipe the output to stdout, rather than a file

            > jupyter nbconvert mynotebook.ipynb --stdout

            PDF is generated via latex

            > jupyter nbconvert mynotebook.ipynb --to pdf

            You can get (and serve) a Reveal.js-powered slideshow

            > jupyter nbconvert myslides.ipynb --to slides --post serve

            Multiple notebooks can be given at the command line in a couple of
            different ways:

            > jupyter nbconvert notebook*.ipynb
            > jupyter nbconvert notebook1.ipynb notebook2.ipynb

            or you can specify the notebooks list in a config file, containing::

                c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

            > jupyter nbconvert --config mycfg.py

To see all available configurables, use `--help-all`.
```